

Two-phase truck-van-drone collaborative transportation with transshipment: A two-stage heuristic approach enhanced with reinforcement learning

Abstract

Urban terminal transportation connects port-based freight flows with urban customers and plays a critical role in the efficiency and sustainability of city logistics systems. Rapid growth in e-commerce and increasing urban density have made such systems more complex, especially when freight must pass through intermediate warehouses and be delivered under heterogeneous accessibility constraints. However, most existing studies on ground-air collaborative delivery focus on simplified urban settings and do not fully capture warehouse transshipment operations or building-level delivery restrictions. This paper studies a truck-van-drone collaborative transportation problem with transshipment (TVDCTP-T), in which shipments are transported between ports and customers via transshipment at warehouses based on coordinated operations of trucks, vans, and drones. High-floor customer requests must be served by drones due to access limitations, whereas low-floor requests can be served by either vans (via couriers) or drones. Flexible docking is also allowed, enabling drones to land on vans different from their launch platforms. To support decision-making in large-scale terminal transportation systems, we develop a reinforcement learning enhanced cascade-based adaptive large neighborhood search algorithm. The solution approach jointly updates van routes and drone sub-routes when customer assignments change, allowing efficient exploration of tightly coupled ground-air decisions. Extensive computational experiments are conducted on both randomly generated instances and realistic cases based on urban logistics operations in Shenzhen. The results show that the proposed approach consistently outperforms benchmark methods in terms of total cost and computational efficiency, particularly for medium- and large-scale instances. [Sensitivity analyses further reveal how request composition, high-floor demand ratio, drone fleet configuration, and time-window tightness influence system performance, providing practical insights for the planning and management of urban terminal transportation systems.](#)

Keywords: truck-van-drone transportation, pickup and delivery problem, flexible docking, ALNS, reinforcement learning, two-stage solution

1. Introduction

Urban terminal transportation is a critical component of modern logistics systems, serving as the interface between large-scale freight networks and urban customers. It typically involves the movement of commodities from ports to inland warehouses, followed by urban distribution to end customers. With the rapid expansion of e-commerce and the spatial dispersion of demand, terminal transportation has become increasingly complex and costly, especially when freight must travel across extended urban corridors and dense city environments. In traditional port-based logistics systems, containers are first transported from ports to warehouses by trucks, after which vans complete customer deliveries. Efficient coordination across these sequential stages is essential to reduce delivery time, operational costs, and traffic congestion, while maintaining reliable and timely service.

In many port-connected metropolitan areas such as Shenzhen, terminal transportation operates through a structured multi-stage process shaped by the rapid growth of cross-border e-commerce and global trade. Due to urban traffic regulations that restrict heavy vehicles from entering dense city centers, containers released from marine terminals are first transported by drayage trucks in batches to suburban transshipment warehouses within predefined time windows. At these facilities, containers are unloaded and parcels are promptly sorted into delivery batches according to service zones and customer time windows, typically within a short turnaround period. Vans are then dispatched to serve designated urban areas. Rather than visiting all customers sequentially, vans operate along planned routes with several intermediate stops determined by access conditions and parking availability in dense environments such as business districts and high-density residential areas. At these stops, drones may be deployed to serve nearby customers within a limited radius, particularly where direct vehicle access is inefficient, such as high-rise buildings. Depending on routing decisions, a van may either wait for the drone or proceed to a downstream stop for rendezvous. In overlapping service areas, drones may also be retrieved by another van, enabling flexible coordination. These operational features are reflected in the proposed model through the integration of multi-stage transportation, van-drone coordination, and flexible rendezvous decisions. To maintain tractability, certain operational details, such as airspace restrictions and real-time traffic variations, are not explicitly modeled.

Optimizing urban terminal transportation can generate substantial economic, environmental, and operational benefits. From an economic perspective, terminal transportation often accounts for a significant

share of total logistics costs due to fragmented demand, frequent stops, and congestion in urban areas. From an environmental standpoint, freight transport contributes considerably to energy consumption and emissions, motivating the integration of low-emission technologies such as drones with conventional ground vehicles. Operationally, coordinated multimodal transportation can improve service reliability and delivery safety, both of which are increasingly important in urban logistics systems. Recent surveys on van-drone routing problems reflect the growing interest in hybrid delivery systems and summarize solution approaches ranging from exact methods to heuristics and metaheuristics (Liang & Luo, 2022; Zhou et al., 2025).

A substantial body of literature has investigated collaborative routing involving heterogeneous vehicles. Extensions of the classical vehicle routing problem have incorporated drones that can be launched from and recovered by trucks or vans under synchronization constraints, demonstrating the potential of ground-air collaboration to improve efficiency. More recent studies consider flexible launch and recovery locations, multi-drone coordination, and dynamic operating environments (Salama & Srinivas, 2022). Parallel to these modeling advances, algorithm-oriented research has increasingly adopted reinforcement learning and other data-driven techniques to address complex routing decisions, particularly under uncertainty or time-dependent conditions (Peng et al., 2025; Peng et al., 2026). Despite these advances, most existing studies focus on simplified delivery settings and often abstract away key operational features of terminal transportation systems. In particular, transfer operations at intermediate warehouses, building-level accessibility constraints (such as distinguishing high-floor and low-floor deliveries), and practical constraints including time windows, vehicle capacities, and flexible docking between vans and drones are rarely considered in an integrated manner. As a result, there remains a gap in models that jointly capture port-to-warehouse transportation, warehouse transshipment, coordinated warehouse-to-customer delivery, and scalable solution approaches suitable for large, realistic terminal networks.

To address these limitations, this study introduces a truck-van-drone collaborative terminal transportation problem with transshipment (TVDCTP-T). The problem explicitly models transfer operations at warehouses, conventional constraints such as time windows and vehicle capacities, and building-level delivery requirements, where high-floor requests must be served by drones while low-floor requests can be completed by either vans (via couriers) or drones. To efficiently solve large-scale instances, we develop a reinforcement learning-enhanced cascade-based adaptive large neighborhood search (RL-C-ALNS) algorithm. The proposed solution approach jointly updates van routes and associated drone sub-routes when

customer assignments change, allowing effective exploration of tightly coupled ground-air decisions. Extensive computational experiments on both randomly generated instances and real-world cases demonstrate that the proposed approach significantly outperforms benchmark methods in terms of solution quality and computational efficiency. The main contributions of this study are summarized as follows:

- This study introduces the truck-van-drone collaborative terminal transportation problem with transshipment (TVDCTP-T), which jointly models port-to-warehouse transport, warehouse transshipment, and coordinated urban delivery. The formulation explicitly captures building-level accessibility constraints (high-floor vs. low-floor requests) together with practical operational requirements such as time windows, vehicle capacities, and flexible van-drone docking. To the best of our knowledge, this is the first to consider such a practical and comprehensive ground-air collaborated transportation problem.
- We develop a tailored reinforcement learning-enhanced cascade-based ALNS (RL-C-ALNS) algorithm that efficiently solves large-scale instances by explicitly modeling the structural dependencies between van routes and drone sub routes via cascade-based operators and guiding the operator selection strategies through RL.
- Extensive experiments on both random instances and real-world instances show the proposed RL-ALNS approach outperforms state-of-the-art benchmarks including Gurobi, greedy, and C-ALNS, regarding multiple evaluation metrics consisting of solution quality, computational efficiency, and operational feasibility. Sensitivity analyses further provide managerial insights into how demand composition, building characteristics, and drone fleet configurations affect terminal transportation performance.

The remainder of this paper is organized as follows: Section presents a comprehensive literature review. Problem description and model formulation are presented in Section 3 and Section 4 respectively. Section 5 elaborates on the proposed solution approach. Section 6 analyzes the results of numerical experiments. The last section concludes the paper.

2. Literature review

We systematically review the related literature from two perspectives: the collaborative truck-drone transportation problem and the transshipment problem.

2.1. Collaborative truck-drone transportation problem

The research on the truck-drone routing problem has attracted the attention of many researchers in recent years. The related research can be traced back to Murray and Chu (2015). They conducted a study in which the truck performs trunk transportation and serves as a mobile launch-and-retrieval platform, while drones act as supplementary last-mile delivery agents. This problem, referred to as the Flying Sidekick Traveling Salesman Problem (FSTSP), allows the drone to serve only a customer per sortie, then return to the truck. Based on the FSTSP, Agatz et al. (2018) further proposed the traveling salesman problem with a drone, in which a truck is equipped with a drone, and the drone is allowed to be launched and retrieved at different customer nodes along the truck route. Compared to the FSTSP, Agatz et al. (2018) did not require drones to return to their starting points, significantly enhancing routing flexibility. They developed several fast route-first, cluster-second heuristics based on local search and dynamic programming. However, configurations involving a single truck and a single drone have limited capacity and struggle to satisfy real-world logistics requirements. Wang et al. (2017) introduced the Vehicle Routing Problem with Drones (VRPD), which considers a fleet of trucks equipped with one or more drones. Their study investigated the problem from a worst-case analysis perspective. Saleu et al. (2022) considered a scenario in which trucks are equipped with multiple drones, where the truck fleet is responsible for standard delivery tasks, while drones carry one package per trip and can perform multiple trips during the planning horizon. They proposed a hybrid metaheuristic solution approach to solve the problem. Salama and Srinivas (2022) introduced non-customer locations referred to as flexible sites, allowing drones to be launched and retrieved at customer locations or flexible sites. They developed an optimization-enabled two-phase search algorithm by hybridizing simulated annealing and variable neighborhood search. Kong et al. (2025) introduced a hybrid autonomous vehicle-drone routing problem for last-mile delivery, where a fleet of autonomous vehicles, vehicle-carried drones, and independent drones collaboratively serve customers to minimize total operational time. Li and Wang (2025) proposed a vehicle-multi-drone collaborative delivery model with payload and endurance constraints and developed an improved genetic algorithm.

Most of the aforementioned studies assume that a drone can serve only one customer per flight, which limits delivery efficiency. Many studies have progressively incorporated more realistic constraints that allow drones to serve multiple customers per trip. Luo et al. (2021) allowed a drone to perform multiple delivery tasks per flight subjecting to endurance constraints. Gonzalez-R et al. (2020) further considered linear

constraints related to endurance and time, and developed an iterated greedy heuristic based on the iterative process of destruction and reconstruction of the solution. However, these studies do not consider the impact of payload variation on drone energy consumption and instead treat drone endurance as a fixed upper bound on flight time or travel distance, which is unrealistic. Liu et al. (2021) incorporated payload-dependent energy consumption into a two-echelon truck-drone routing problem, allowing a drone to deliver multiple parcels within a single flight, and developed a hybrid heuristic integrating nearest neighbor and cost-saving strategies. Jiang et al. (2025) developed a hybrid algorithm that combines variable neighborhood search and simulated annealing to solve a problem in which multiple trucks carry heterogeneous drones to perform multi-parcel deliveries, while explicitly considering the impact of payload on energy consumption. In this study, we consider the effect of parcel weight on drone energy consumption in order to enhance the practical applicability of the model.

In some studies, trucks are considered as launch platforms for drones and do not take part in the delivery process. The model proposed by Poikonen and Golden (2020) separates drone launch nodes from customer nodes, where the truck directly travels to the drone retrieval node after launching the drone and is not allowed to visit other nodes. They developed a flexible heuristic solution. Considering a heterogeneous truck-drone cooperative delivery scenario, Kang and Lee (2021) assumed that each drone performs a single delivery task and returns to its truck, while the truck waits at the node until the drone returns. They proposed an exact algorithm based on the logic-based Benders decomposition approach. Zhang et al. (2021) considered a post-disaster relief scenario, in which trucks serve as launch bases for drones, and proposed a heuristic algorithm based on column generation. In the study by Ghiasvand et al. (2024), trucks transport two kinds of commodities to a distribution center, from which drones depart to serve customers. Allowing trucks to actively participate in delivery operations can significantly enhance transportation efficiency. On the contrary, little research has focused on truck-drone collaborative delivery systems, where both trucks and drones serve customers. Kuo et al. (2023) introduced a collaborative delivery model for trucks and drones, aiming to minimize both the makespan and carbon emissions. They employed the non-dominated sorting genetic algorithm II (NSGA-II) to address the resulting multi-objective optimization problem. In addition, some studies have introduced flexible docking mentioned above, which allows a drone to return to any truck after completing its delivery task, thereby improving routing flexibility. Xing et al. (2024) and Xiao and Gao (2024) allow drones to return to any truck after delivery, but each drone flight is still restricted to serving

only one customer. On this basis, K. Li et al. (2025) studied a dynamic truck-drone collaborative routing problem under hurricane-induced wind uncertainty, in which drones can flexibly launch from and land on different trucks. Fan et al. (2026) formulated a multi-depot electric vehicle routing problem with drones considering time-dependent traffic, where multiple depots dispatch EVs and drones cooperatively with open strategies to serve customers. In contrast to the abovementioned works, this study not only considers flexible docking but also allows a drone to perform multiple delivery tasks within a single flight, subject to energy consumption constraints.

Only a few studies have considered simultaneous pickup and delivery in drone-involved transportation scenarios. Karak and Abdelghany (2019) considered pickup and delivery in the TSP-D, where all customers were served by drones, while the truck acted solely as a platform for launching and retrieving drones. Jiang et al. (2024) investigated the fulfillment of pickup and delivery demands in rural areas using a truck-drone fleet and proposed a tailored adaptive large neighborhood search metaheuristic. Ren et al. (2025) developed an electric vehicle-drone pickup and delivery model with time-dependent vehicle speeds and designed an offline-online heuristic algorithm to solve the problem. Duan et al. (2026) introduced a multi-objective time-dependent vehicle routing problem with drone considering simultaneous pickup and delivery, and proposed a multi-objective hyper heuristic memetic algorithm. Zeng et al. (2026) investigated a multi-truck and multi-drone collaborative pickup and delivery problem with service time windows and spatiotemporal no-fly zones, and proposed an adaptive large neighborhood search to minimize total operational cost.

In summary, this study incorporates practical factors such as multi-visit operations of drones, flexible docking, and open routing. In addition, our study extends beyond a single last-mile delivery stage by considering coordination across multiple delivery stages and transportation modes, including container drayage by trucks and pickup and delivery by van-drone collaboration to improve overall system performance. Compared with existing single-stage truck-drone or van-drone models, the distinguishing feature of the proposed TVDCTP-T model lies in the temporal and commodity synchronization across multiple transportation stages. Specifically, the problem extends beyond the conventional "last-mile" delivery scope by capturing the tight coupling between port-to-warehouse container drayage and warehouse-to-customer urban distribution, thereby introducing additional spatiotemporal synchronization constraints inherent to multi-stage terminal transportation. Several modeling elements in the framework, such as flexible van-drone docking, open routing, simultaneous pickup and delivery, and time windows, are commonly

studied extensions in collaborative delivery systems. These components improve modeling realism but are not specific to terminal transportation. They can be relaxed or adapted in other application contexts without altering the core structure of multi-stage coordination. Beyond the specific port-connected setting, the proposed framework can be interpreted as a general representation of multi-tier logistics systems. In multimodal transport systems, the upstream stage may correspond to trunk transportation (e.g., port or rail hubs), while the downstream stage reflects regional or urban distribution. The coordination mechanism between stages remains applicable in settings where synchronization of schedules and capacity allocation is required across different transport layers. In e-commerce and express delivery systems, the model can also be related to integrated distribution networks involving depots, vehicle fleets, and drone-assisted delivery. The heterogeneous service requirements captured in the model, such as building accessibility differences, reflect common operational constraints in dense urban logistics environments. The flexible docking mechanism further represents dynamic coordination among delivery resources under congestion and limited access conditions. With minor structural adjustments, such as removing the upstream drayage stage or simplifying pickup-delivery coupling into pure delivery operations, the model can be reduced to standard last-mile van-drone routing problems. This demonstrates that the proposed framework is sufficiently general while still capturing the essential coordination structure of terminal transportation systems.

2.2. Transshipment problem

In practical logistics operations, transshipment occurs between ports, yards, and warehouses. De Rosa et al. (2002) introduced transshipment into the classical Capacitated Arc Routing Problem (CARP) in the context of urban waste collection and proposed the Arc Routing and Scheduling Problem with Transshipment (ARPT). In their model, dedicated collection vehicles pick up waste along their routes and, once reaching capacity, travel to a transshipment station to unload. The waste is then transported to the final disposal site by high-capacity trucks. The authors constructed integer linear programming relaxation-based lower bounds and designed a tabu search heuristic to solve the problem. Perboli et al. (2011) proposed a two-echelon capacitated vehicle routing problem (2E-CVRP), in which commodities are transshipped through satellite depots, and developed two math-heuristic algorithms to solve the problem.

Research on port transshipment operations has mainly focused on the operational equipment at transshipment centers, including yard layout planning, rail-mounted gantry cranes, and automated guided

vehicles (AGVs). These studies emphasize the optimization of operational processes, with the aim of improving coordination efficiency across different transport stages both within and beyond the port, and enhancing the turnover speed of intermodal loading units (ILUs). Chen et al. (2020) investigated the coordinated scheduling of AGVs and automated cranes in automated container terminals, aiming to optimize container transport efficiency across different operational zones within the port hub. S. K. Chen et al. (2021) focused on on-land transshipment between two container terminals in Singapore, and proposed an autonomous truck scheduling problem with platooning and speed optimization, formulated as a Mixed-Integer Second-Request-Cone Programming (MISOCP) model and developed a column generation-based heuristic algorithm. Cheng et al. (2025) studied the coordinated scheduling of internal port trucks under a dual-cycle operation strategy to reduce quay crane waiting times, and developed a graph-modeling approach integrated with Double Deep Q-Networks (DDQN).

At the last-mile logistics level, research on transshipment typically focuses on the flow of commodities from warehouses to vehicles, which is a common feature in urban courier networks and warehouse-to-local depot systems. With the increasing scale and complexity of logistics systems, transshipment research has become more diversified, encompassing multiple stages such as inventory management and distribution (Avci & Yildiz, 2020). For instance, Shen and Honda (2009) developed an integrated replenishment and routing plan for a three-echelon supply chain system consisting of a single plant, multiple distribution centers, and multiple retailers. Their study considered lateral transfers of both inventory and vehicles between distribution centers, with the objective of minimizing transportation cost through coordinated pooling strategies. Achamrah et al. (2022) studied a dynamic and stochastic inventory routing problem (DSIRP) in a two-level supply chain to mitigate customer-level shortages under uncertainty. The problem was formulated as a multi-product DSIRP with stochastic demand, and solved using a hybrid resolution approach combining mathematical modelling, Genetic Algorithm, and Deep Reinforcement Learning (Deep Q-learning). Friedrich and Elbert (2022) considered different problem variants of the vehicle routing problem with transshipment facilities (VRPTF) in city logistics, where customer demands can be delivered either directly or via third-party transshipment facilities such as urban consolidation centers. Yu et al. (2020) proposed a network in which a single warehouse supports delivery tasks performed by trucks carrying multiple robots, and developed heuristic and hybrid metaheuristic algorithms. On this basis, recent studies have further examined transshipment scenarios involving multiple warehouses (J. Li et al., 2025; Yu et al., 2023). C. Chen

et al. (2021) introduced a vehicle routing problem with time windows and delivery robots (VRPTWDR), in which vehicles equipped with several self-driving delivery robots serve nearby customers in a two-echelon network. Alfandari et al. (2022) addressed a last-mile delivery problem in which a vehicle transports parcels from a central depot to facilities, from which autonomous robots are launched to perform deliveries, and proposed an efficient branch-and-Benders-cut scheme.

2.3. Comprehensive comparison of existing works

Table 1 presents a comparison of the literature relevant to this study. Unlike existing research, our problem simultaneously considers time windows, open routes, and pickup and delivery services, executed collaboratively by trucks and drones capable of performing multiple delivery tasks.

Incorporating time window constraints requires vehicles to deliver customer requests within specified periods, introducing numerous complex constraints into the model. Dellaert et al. (2019) introduced time window synchronization constraints into their model and developed branch-and-price-based algorithms. Recently, J. Li et al. (2025) examined collaborative distribution in agricultural wholesale markets under time window constraints, and proposed a two-stage iterative algorithm combining branch-and-bound with adaptive large neighborhood search.

In this study, trucks and drones can perform open routes, meaning that vehicles do not need to return to the departing warehouse. This avoids the extra travel cost caused by mandatory returns, improves system flexibility, and reduces transportation costs. However, few studies have considered this. Ensafian et al. (2023) introduced open routes into a two-echelon vehicle routing problem (2EVRP) without considering time constraints, allowing couriers and Autonomous Mobile Lockers (AMLs) not to return to the origin, and proposed an Adaptive Backtracking-Simulated Annealing metaheuristic (ABSM). Similarly, Triantali et al. (2025) ignored time constraints, allowing fleets and crowd-sourced drivers to have different start and end points, and designed a dedicated metaheuristic based on the variable neighborhood search. In contrast, our study considers pickup and delivery services in one trip.

Considering simultaneous pickup and delivery is another key feature of this study. In real logistics systems, delivery vehicles often perform both pickup and delivery tasks simultaneously; limiting vehicles to either pickup or delivery, or allowing only a single task per trip would result in resources underutilization. However, in the literature, few studies considered simultaneous pickup and delivery in transshipment

problems, such as Ensafian et al. (2023); Jiang et al. (2024); J. Li et al. (2025). Ensafian et al. (2023) did not consider time-related constraints; Jiang et al. (2024) focused solely on a single last-mile transport problem without involving multi-stage transshipment coordination; and in the study of J. Li et al. (2025), vehicles could only return to the originating warehouse after completing deliveries. Our study further extends the model by integrating temporal coordination, open routing, and multi-stage transshipment operations.

The diversification of delivery modes has also become an important trend in recent research. Namely, studies are no longer limited to traditional fleet-based deliveries but increasingly consider crowd-shipping and delivery robots. Yu et al. (2023) investigated a two-echelon delivery network in which city freighters and occasional drivers (ODs) perform tasks, and developed an efficient hybrid adaptive large neighborhood search (HALNS) algorithm. Similarly, Nguyen and Hop (2025) considered deliveries by both crowd-shipping and dedicated vehicles, and a novel hierarchical heuristic algorithm was developed to classify customers by appropriate vehicle types. Delivery robots can be categorized into ground-based robots and drones. In the study of Ostermeier et al. (2022), trucks serve as carriers for low-speed robots that navigate on sidewalks at pedestrian speed. The trucks travel to robot depot locations, launch robots to perform deliveries, and pick them up upon return. The authors designed both exact algorithms and heuristics for this problem. Li et al. (2021) proposed the coordination of AMLs and couriers to achieve synchronous pickup and delivery in a two-echelon network, effectively addressing the frequent returns to fixed warehouses in traditional delivery. They developed a hybrid Clark and Wright heuristic (HCWH) and an enhanced HCWH (EHCWH). Drones, as an emerging mode of transportation, can deliver parcels more quickly, consume less energy, and reduce delivery costs. Compared with ground-based robots, drones are more maneuverable and are not constrained by road structures or traffic conditions. However, only a few studies in the context of transshipment have investigated truck-drone collaborative delivery, such as Morim et al. (2024) and Zhen et al. (2025). Morim et al. (2024) proposed a last-mile delivery problem that integrates truck-drone collaboration with robot stations, and developed a General Variable Neighborhood Search metaheuristic (GVNS). In their study, trucks can collaborate with drones for deliveries or travel to robot stations, where robots complete deliveries to customers. Zhen et al. (2025) studied a system in which drones pick up requests from the origin, transfer the commodities to trucks for line-haul transport, and then perform final deliveries near the destination. Clearly, these studies differ from the present research. In our study, container trucks pick up request containers from the origin to a transshipment warehouse, after which request allocation is

performed, and a van-drone fleet collaboratively completes last-mile deliveries. Drones undertake part of the short-distance delivery tasks, which not only reduces labor costs but also significantly shortens last-mile delivery time.

Regarding solution methods, most existing studies adopt heuristic algorithms, with only a few using exact methods such as branch-and-bound. Exact algorithms are slow for large-scale instances, while heuristics can solve large problems but may yield low-quality solutions. To address these limitations, we carefully design a solution approach combining machine learning and heuristics to efficiently solve large-scale instances.

To fill the abovementioned research gaps, our study proposes the container truck-van-drone collaborative transportation problem, considering features such as multi-task execution, temporal synchronization, open routes, time windows, and pickup and delivery services, which are collaboratively performed by drones and trucks capable of executing multiple delivery tasks. By minimizing waiting time costs and operational costs, the system effectively improves overall delivery efficiency.

300

301

Table 1 Summary and comparison on related literature.

Study	Vehicle types	Transshipment constraints	Multi-depot	Pickup and delivery	Time window	Flexible docking	Open route	Objectives	Solution
Agatz et al. (2018)	T, D							TT	Exact, Heuristic
Alfandari et al. (2022)	R		√		√			MT+ST+NLD	Exact
Ostermeier et al. (2022)	R		√		√			TS+TC	Exact, Heuristic
Yu et al. (2022)	T, CF, O	T-CF	√					TC+TS	Heuristic
Ensafian et al. (2023)	C		√	√			√	TS+TC	Heuristic
Yu et al. (2023)	T, O	T-T	√		√			TC+CC+CF	Heuristic
Jiang et al. (2024)	V, D			√	√	√		TC	Heuristic
Lehmann and Winkenbach (2024)	V	V-V	√	√	√			TC+TS	Exact, Heuristic
Triantali et al. (2025)	T, O	T-T	√				√	EC+EE+TC	Heuristic
Nguyen and Hop (2025)	T, O	T-T	√		√			TC+HT+CF	Heuristic
Hou et al. (2025)	V		√		√			N+TC	Heuristic
Jiang et al. (2025)	V, D				√			TT	Heuristic
Ren et al. (2025)	V, D			√	√			P	Heuristic
Kong et al. (2025)	ADV, D							TT	Heuristic
Li and Wang (2025)	V, D							TT	Heuristic
Liu et al. (2025)	V, D							TT	Heuristic
K. Li et al. (2025)	V, D			√	√	√		P	Heuristic
Duan et al. (2026)	V, D			√	√			TC+TTV	Heuristic
Zhen et al. (2025)	V, D			√	√	√		TC+PC	Exact
Fan et al. (2026)	EV, D		√		√	√	√	TC+TS	Heuristic
Zeng et al. (2026)	V, D			√	√			TC+ECC+LC	Heuristic
J. Li et al. (2025)	V	V-V	√	√	√			TS+TC+HT	Heuristic
Our Study	T, Van, D	T-Van	√	√	√	√	√	TS+TC+WT	AI+ Heuristic

302

303

304

305

T: truck; V: vehicle; D: drone; O: occasional drivers; C: courier; R: robot; Transshipment Mode: for example, T-D refers to the transfer of commodities from truck to drone; TT: total time; TC: transportation cost; TS: total setup cost; CC: connection costs; HT: handling cost; CF: compensation fee; ADV: autonomous delivery vehicle; EV: electric vehicle; N: number of vehicles; EC: expected cost of occasional driver; ECC: energy consumption cost; LC: labor cost; EE: expected cost of emergency vehicles; MT: max tardiness; ST: sum tardiness; NLD: number of late deliveries; P: profit; PC: penalty cost for delayed deliveries; TTV: total time window violation; WT: waiting time.

3. Problem description

We consider an operator that owns a fleet of homogeneous tractors K , trailers P , vans V , and drones D , respectively, aiming to address a two-phase pickup and delivery problem in a logistics distribution network regarding a set of requests R that must be served. In this study, a container drayage truck (hereafter referred to as a truck) consists of a tractor and a trailer, where containers are loaded onto trailers and transported by tractors. Trailers cannot operate independently and must be hauled by tractors, while tractors can travel alone if necessary. The first phase focuses on transporting commodities between ports and warehouses via container drayage trucks, while the second phase concentrates on pickups and deliveries between warehouses and customers using vans and drones. During this process, transshipment occurs at warehouses where commodities from containers need to be transferred to vans. The objective is to minimize the total transportation cost and vehicle waiting time by jointly optimizing the decisions of vehicle assignments and routings across different modes.

We model the TVDCTP-T on a graph $G = (N, A)$, where N denotes the set of nodes, and A represents the set of arcs connecting these nodes. The node set includes a set of tractor depots N_m , trailer depots N_n , port nodes O , warehouses N_s , and customer nodes N_c , respectively. Each arc $(i, j) \in A$ is characterized by a travel distance and a corresponding travel time. Each container $c \in C$ is handled by container drayage carried out via a fleet of homogeneous tractors K and homogeneous trailers P . Customer services are provided by a cooperation between trucks, vans, and drones. Accordingly, tractors and trailers are allowed to travel among $N_0 = N_n \cup N_m \cup O \cup N_s$, where vans and drones operate exclusively between warehouses and customer nodes, i.e., $N_+ = N_s \cup N_c$.

In this study, we allow requests with two directions: (i) import direction: from port/warehouses to customers, and (ii) export direction: from customers to port, termed as import requests and export requests, respectively. Each request $r \in R$ is uniquely associated with a customer $i \in N_c$ and a prematched container $c \in C$. Each customer $i \in N_c$ is associated with a building-floor attribute $\tilde{h}_i$, a delivery demand d_i , a pickup demand p_i , and a service time window (e_i, l_i) that are not allowed to be violated. Each customer must be served exactly once by either a van or a drone within the specified time window. Due to the substantial labor and time required for serving high-floor buildings, high-floor requests ($\tilde{h}_i = 1$) are mandatorily served by drones, while low-floor requests ($\tilde{h}_i = 0$) may be served by either vans or drones.

In practice, for import directions, commodities may originate not only from the ports, but also from warehouses because warehouses may have stored commodities in advance or possess remaining inventories. For time windows, it restricts merely the delivery time at customers for import requests, excluding the arrival time or departure time at the warehouse; while it only restricts the pick up time at customers for export requests. For practicality, we consider three types of requests: (i) pure import requests that only need to deliver the commodities to the customer; (ii) pure export requests that only need to collect the commodities from the customer; and (iii) combined requests that contains both export and import demands, i.e., the vehicle delivers the commodities to the customer and collects the customer's export commodities immediately after. To fulfill a request, the capacity requirement of the vehicle must be satisfied during the entire trip. Please note that this study focuses on urban transportation of commodities that ends mainly at developed areas instead of rural areas. Therefore, in this study, we do not consider transportation from port to customers directly without using warehouses for transshipment, which can be incorporated in future studies.

Each container $c \in C$ has a predefined origin but no predetermined destination. The destination warehouse of a container is determined endogenously through routing decisions. Containers loaded with delivery requests are loaded onto a trailer and then transported by tractors to selected warehouses, where unloading operations are conducted and require a service time s_c . For flexibility, we consider a drop-and-go style for container drayage, where tractors are allowed to leave after unloading the laden trailer without wasting time waiting for unload. After all delivery containers have been completely unloaded, tractors and trailers return to their respective depots. Subsequently, after vans and drones complete their routes and return collected pickup items to warehouses for consolidation, containers loaded with pickup commodities are transported from warehouses back to the port by tractors hauling trailers.

All container movements between pickup points and warehouses must be performed by tractors equipped with trailers. A tractor may travel independently, whereas a trailer cannot operate without being hauled. Each tractor departs from a tractor depot N_m , travels to a trailer depot N_n to attach a trailer, and can haul at most one trailer at a time.

Vans V and drones D deliver commodities unloaded from containers at warehouses to customers and transport collected pickup items back to warehouses upon completing their routes. Each van has a maximum load capacity C_v and can carry multiple drones during its trip. Vans may launch one or multiple drones either at warehouses or at customer nodes. Subject to the drone payload capacity C_d and flight endurance

$\tilde{e}_d$, a drone may serve multiple customers before landing, with energy consumption depending on payload weight and flight distance. While drones are performing delivery tasks, vans may continue serving other customers instead of waiting for drone retrieval. In addition, we consider flexible docking between vans and drones, meaning that a drone may land on a different van than the one it departed from. Drone launching and rendezvous docking are permitted at both transshipment warehouses and customer nodes. During a single continuous flight, a drone can serve multiple customers subject to its payload and endurance limits; however, multiple docking events within the same mission are strictly prohibited, meaning that each deployed drone docks exactly once at the end of its flight.

After completing deliveries, drones may rendezvous with any nearby van to unload parcels and replace batteries; however, each van can carry at most three drones. Specifically, if a drone arrives at a rendezvous node earlier than the van, it performs a safe landing to conserve energy and waits for the van to arrive. Vans and drones are allowed to wait for each other at rendezvous nodes. Since the time required for drone launching, recovery, and battery replacement is commonly neglected in routing studies due to its limited impact on system-level performance (Jiang et al., 2024; Xiao and Gao, 2024), these operations are omitted in this study. In large-scale terminal transportation systems, routing decisions are primarily driven by travel times, customer distribution, and synchronization requirements between vans and drones. Compared with inter-node travel times, these short handling operations tend to have limited influence on routing structure and assignment decisions. From a cost perspective, ignoring these operations may slightly underestimate total cost, particularly when drone dispatching is frequent. However, such effects are generally proportional across feasible solutions and have limited impact on the relative comparison of routing strategies. Consequently, omitting launch, recovery, and battery replacement times has limited impact on the coordination patterns or feasibility of van-drone interactions, which are primarily determined by travel times and service constraints. We acknowledge the boundaries of these assumptions. In localized scenarios characterized by extremely frequent, ultra-short-distance drone operations, or highly constrained battery resources, the cumulative effect of these setup times may become a non-negligible operational bottleneck. Under such extreme operating regimes, these temporal dynamics should be explicitly modeled in future research. After completing their service tasks, vans and drones may return to warehouses different from their starting locations. The total travel distance of each van and drone must not exceed its maximum allowable travel distance $\tilde{K}_i$.

Containers are transported to warehouses by tractors hauling trailers. After unloading, the commodities are transferred to vans for customer service. This transshipment process requires synchronization between the two transportation stages in both commodity flow and timing. In terms of commodity flow, only vans and drones associated with the warehouse receiving a given container are permitted to serve the corresponding customer requests. In terms of timing, vans and drones may depart from a warehouse only after the unloading operation of the relevant container is completed. Similarly, for export requests collected by vans and drones, tractors are allowed to transport containers back to the port only after all pickup commodities have been returned to the warehouse and loaded into containers.

The TVDCTP-T problem is illustrated in Figure 1. Tractors and trailers are responsible for container transportation, where some containers loaded with delivery requests are transported from the port to warehouses. In addition, certain delivery requests are supplied by warehouses located far from customers; in such cases, containers may be transferred between warehouses before final delivery by vans and drones. After container unloading, vans and drones at warehouses depart to perform pickup and delivery services. Vans may launch drones at customer nodes and then proceed to subsequent customers. If no drone is launched, the van carries the drones to the next node. Pickup commodities collected by vans and drones are loaded into containers at warehouses and finally transported back to the port by container drayage.

Besides, we present another example to illustrate the service sequences and the time windows, as shown in Figure 2, Van 1 unloads the delivery commodities and loads the pickup commodities during a single visit, and the entire operation must be completed within the customer's specified time window [110, 195]. This example explicitly demonstrates the flexible drone docking mechanism across different vans. As shown, a drone represented by the dashed line is launched from Van 1 at Node 3. It then sequentially serves Node 5, a delivery-only request marked with a white circle, and Node 6, a pickup-only request marked with a grey circle. Instead of returning to Van 1, the drone flies to Node 7, where it is successfully recovered by Van 2, which is represented by the grey solid line.

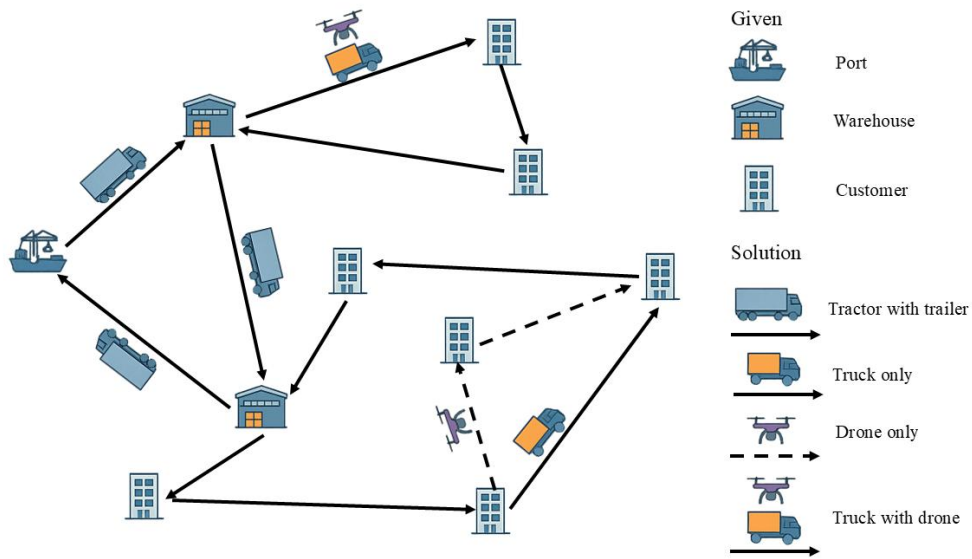


Figure 1 An example of truck-van-drone transportation routes in TVDCTP-T.

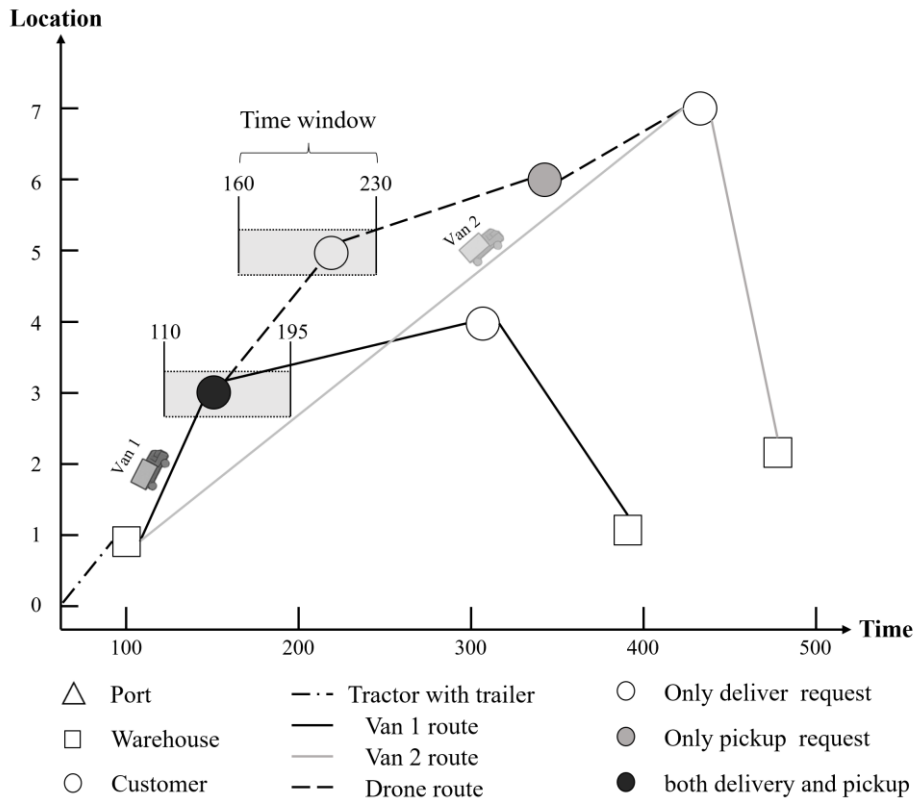


Figure 2 An illustrative time-space diagram of the service routes.

Table 2 Notations and definitions.

Notation	Definition	Used in Stage 1	Used in Stage 2
Sets			
R	Set of requests	✓	

N_m	Set of tractor depots nodes	✓	✓
N_n	Set of trailer depots nodes	✓	✓
O	Set of port nodes	✓	✓
N_s	Set of warehouse nodes	✓	✓
N_c	Set of customer nodes	✓	
N	Set of all nodes, $N = O \cup N_s \cup N_n \cup N_m \cup N_c$	✓	
N_0	Set of nodes for the container drayage phase, $N_0 = O \cup N_s \cup N_n \cup N_m$	✓	✓
N_+	Set of nodes for the pickup and delivery phase, $N_+ = N_s \cup N_c$	✓	
C	Set of containers	✓	
K	Set of tractors	✓	✓
P	Set of trailers	✓	✓
V	Set of vans	✓	
D	Set of drones	✓	
A_1	Set of arcs (i, j) with $i, j \in N_0$	✓	✓
A_2	Set of arcs (i, j) with $i, j \in N_+$	✓	

Parameters

c_i	Travel cost per unit distance for vehicle i , $i \in K \cup V \cup D$	✓	✓
$\tilde{d}_{ij}$	Distance between node i and node j	✓	
β_i	Fixed cost for dispatching vehicle i , $i \in K \cup V \cup D$	✓	✓
α	Penalty cost per unit waiting time occurred during transshipment	✓	✓
o_c	Origin node of container $c \in C$, $o_c \in O \cup N_s$	✓	✓
$\tau_{i,j}^k$	Travel time of tractor k on arc (i, j)	✓	✓
$\tau_{i,j}^v$	Travel time of van v on arc (i, j)	✓	
$\tau_{i,j}^d$	Travel time of drone d on arc (i, j)	✓	
s_c	Loading/unloading time of container c	✓	✓
e_i/l_i	Earliest and latest service times of customer i	✓	
$\hat{e}_c$	Earliest service time of container c in the second stage		✓
d_i	Delivery demand of customer i	✓	
p_i	Pickup demand of customer i	✓	
$\tilde{c}_i$	Capacity of vehicle i , $i \in V \cup D$	✓	
e_{ij}^d	Energy consumption of drone d traversing arc (i, j)	✓	
$\tilde{e}_d$	Battery capacity of drone d	✓	
β_1	Power consumption coefficient per unit payload demand	✓	
β_0	Power consumption coefficient required for hovering	✓	
$\tilde{k}_i$	Maximum travel distance of vehicle i , $i \in K \cup P \cup V \cup D$	✓	✓
$\tilde{h}_j$	Binary parameter, 1 for high-floor request; 0 for low-floor request	✓	

Decision Variables

$x_{i,j}^v$	Binary variable, 1 if van $v \in V$ travels from node $i \in N_+$ to node $j \in N_+$; 0 otherwise	✓	
$y_{i,j}^k$	Binary variable, 1 if tractor $k \in K$ travels from node $i \in N_0$ to node $j \in N_0$; 0 otherwise	✓	✓
$u_{i,j}^{k,p}$	Binary variable, 1 if tractor $k \in K$ carries trailer $p \in P$ when traveling from node $i \in N_0$ to node $j \in N_0$; 0 otherwise	✓	✓

$v_{i,j}^d$	Binary variable, 1 if drone $d \in D$ voyage from node $i \in N_+$ to node $j \in N_+$; 0 otherwise	✓	
$g_{i,j}^{vd}$	Binary variable, 1 if van $v \in V$ carries drone $d \in D$ when traveling from node $i \in N_+$ to node $j \in N_+$; 0 otherwise	✓	
k_i^{vd}	Binary variable, 1 if van $v \in V$ launches or retrieves drone $d \in D$ at node $i \in N_+$; 0 otherwise	✓	
$\tilde{u}_i$	Binary variable, 1 if vehicle $i \in K \cup P \cup V \cup D$ is utilized; 0 otherwise	✓	✓
o_{cr}^v	Binary variable, 1 if request $r \in R$ is transferred from container $c \in C$ to van $v \in V$; 0 otherwise	✓	
z_{cp}	Binary variable, 1 if container $c \in C$ is assigned to trailer $p \in P$; 0 otherwise	✓	✓
r_{cs}	Binary variable, 1 if container $c \in C$ transship commodities at warehouse $s \in N_s$; 0 otherwise	✓	
m_{cv}	Binary variable, 1 if van $v \in V$ loads commodity from container $c \in C$; 0 otherwise	✓	
D_{iv}	Remaining delivery load on van v upon arrival at node $i \in N_+$	✓	
P_{iv}	Collected pickup load on van v upon arrival at node $i \in N_+$	✓	
D_{id}^- / D_{id}^+	Remaining delivery load of drone d upon arrival at / departure from node $i \in N_+$	✓	
P_{id}^- / P_{id}^+	Collected pickup load of drone d upon arrival at / departure from node $i \in N_+$	✓	
a_i^k / d_i^k	Arrival/departure times of tractor k at node $i \in N_0$	✓	✓
a_i^v / d_i^v	Arrival/departure times of van v at node $i \in N_+$	✓	
a_i^d / d_i^d	Arrival/departure times of drone d at node $i \in N_+$	✓	
h_c	Completion time of loading/unloading container c at a warehouse	✓	
H_c	Completion time of unloading container c at the port		✓
e_{id}^- / e_{id}^+	Cumulative energy consumption of drone d upon arrival at / departure from node $i \in N_+$	✓	

4. Mathematical modeling

The TVDCTP-T problem is formulated as a mixed-integer linear programming (MILP) model. The objective (1) is to minimize the total cost incurred by all vehicles, including transportation costs, fixed costs, and time-related penalty costs.

$$\begin{aligned}
 \min F = & c_K \sum_{k \in K} \sum_{i \in N_0} \sum_{j \in N_0} \tilde{d}_{ij} * y_{ij}^k + \sum_{i \in K \cup V \cup D} \beta_i * \tilde{u}_i + \alpha \sum_{i \in N_s} \sum_{k \in K} (d_i^k - a_i^k) + c_V \sum_{v \in V} \sum_{i \in N_+} \sum_{j \in N_+} \tilde{d}_{ij} * x_{ij}^v \\
 & + c_D \sum_{i \in N_+} \sum_{j \in N_+} \sum_{d \in D} \tilde{d}_{ij} * v_{ij}^d
 \end{aligned} \tag{1}$$

s.t.

$$\sum_{j \in N_0} y_{i,j}^k = \tilde{u}_k, \forall k \in K, i \in N_m \quad (2)$$

$$\sum_{j \in N_0} y_{j,i}^k = \tilde{u}_k, \forall k \in K, i \in N_m \quad (3)$$

$$\sum_{j \in N_0} y_{i,j}^k = \sum_{j \in N_0} y_{j,i}^k, \forall k \in K, i \in N_0 \quad (4)$$

$$\sum_{j \in N_0} \sum_{k \in K} u_{i,j}^{k,p} = \tilde{u}_p, \forall p \in P, i \in N_n \quad (5)$$

$$\sum_{j \in N_0} \sum_{k \in K} u_{j,i}^{k,p} = \tilde{u}_p, \forall p \in P, i \in N_n \quad (6)$$

$$\sum_{k \in K} \sum_{j \in N_0} u_{i,j}^{k,p} = \sum_{k \in K} \sum_{j \in N_0} u_{j,i}^{k,p}, \forall p \in P, i \in N_0 \quad (7)$$

Constraints (2) and (3) ensure that each tractor starts its route from the tractor depot and returns to the same depot at the end of its route. Constraints (4) guarantee tractor flow conservation. Constraints (5) and (6) guarantee that each trailer departs from the trailer depot and returns to the trailer depot. Constraints (7) ensure the flow conservation constraint for trailers.

$$u_{i,j}^{kp} \leq y_{i,j}^k, \forall k \in K, p \in P, i, j \in N_0 \quad (8)$$

$$\sum_{i \in N_0} \sum_{j \in N_0} y_{i,j}^k * \tilde{d}_{ij} \leq \tilde{K}_k * \tilde{u}_k, \forall k \in K \quad (9)$$

$$\sum_{k \in K} \sum_{i \in N_0} \sum_{j \in N_0} u_{i,j}^{k,p} * \tilde{d}_{ij} \leq \tilde{K}_p * \tilde{u}_p, \forall p \in P \quad (10)$$

$$\sum_{i \in N_0} y_{i,j}^k \leq 1, \forall k \in K, j \in N_s \quad (11)$$

Constraints (8) characterize the collaborative relationship between tractors and trailers along the routes: tractors are allowed to travel independently without trailers, whereas trailers can only move when attached to tractors. Constraints (9) and (10) impose the maximum travel distance limits on tractors and trailers, respectively. Constraints (11) restrict that each tractor can visit the same transshipment facility at most once.

$$\sum_{p \in P} z_{cp} = 1, \forall c \in C \quad (12)$$

$$\sum_{i \in N_s} r_{ci} = 1, \forall c \in C \quad (13)$$

$$\sum_{i \in N_s \cup N_n} \sum_{k \in K} u_{i,o_c}^{k,p} = z_{cp}, c \in C, p \in P \quad (14)$$

$$\sum_{k \in K} u_{o_c,j}^{k,p} = z_{cp} * r_{cj}, \forall j \in N_s, c \in C, p \in P \quad (15)$$

$$\sum_{j \in N_0} \sum_{k \in K} u_{i,j}^{k,p} \geq z_{cp} * r_{cj}, \forall i \in N_s, c \in C, k \in K, p \in P \quad (16)$$

Constraints (12) and (13) require that all containers must be transported by trailers, and each container is uniquely assigned to one transshipment facility as its destination. Constraints (14) ensure that a tractor, together with an attached trailer, travels from other nodes to the origin node of the corresponding container. Constraints (15) describe the transportation of containers from their origin nodes to the selected transshipment facilities by tractors carrying trailers. Constraints (16) represent the movement of trailers after

completing the container transportation tasks.

$$a_j^k \geq d_i^k + \tau_{i,j}^k - M(1 - y_{i,j}^k), \forall k \in K, (i, j) \in A_1 \quad (17)$$

$$a_j^k \leq d_i^k + \tau_{i,j}^k + M(1 - y_{i,j}^k), \forall k \in K, (i, j) \in A_1 \quad (18)$$

$$d_j^k \geq a_j^k, \forall k \in K, j \in N_0 \quad (19)$$

$$h_c \geq a_j^k + s_c - M(1 - u_{o_c,j}^{k,p}) - M(1 - r_{cj}) \forall k \in K, p \in P, c \in C, j \in N_0 \quad (20)$$

$$d_i^k \geq h_c - M(1 - \sum_{j \in N_0} u_{i,j}^{k,p}) - M(1 - r_{ci}) - M(1 - z_{cp}), \forall k \in K, i \in N_s, p \in P, c \in C \quad (21)$$

Constraints (17) - (19) describe the time propagation relationship for a tractor traveling from node i to j . Constraints (20) update the completion time of container unpacking operations at transshipment warehouses. Constraints (21) ensure that the departure time of a tractor carrying a trailer from a transshipment warehouse is not earlier than the completion time of the corresponding container unpacking operations.

$$\sum_{v \in V} o_{ir}^v = 1, \forall r \in R, i \in C \quad (22)$$

$$\sum_{v \in V} m_{cv} \geq 1, \forall c \in C \quad (23)$$

$$o_{cr}^v \leq m_{cv}, \forall v \in V, c \in C, r \in R \quad (24)$$

$$M * \sum_{r \in R} o_{cr}^v \geq m_{cv}, \forall v \in V, c \in C, r \in R \quad (25)$$

$$d_i^v \geq h_c - M(2 - r_{ci} - m_{cv}), \forall v \in V, \forall c \in C, i \in N_s \quad (26)$$

$$D_{iv} = \sum_{r \in R} \sum_{c \in C} r_{ci} * o_{cr}^v * d_r, \forall v \in V, i \in N_s \quad (27)$$

Constraints (22) and (23) require that all containers must be loaded by vans, while allowing different requests within the same container to be loaded and delivered by different vans. Constraints (24) and (25) illustrate the relation of o_{cr}^v and m_{cv} . Constraint (26) ensures that the departure time of a van from the warehouse is later than the unpacking completion times of the containers it carries. Constraint (27) describes the load update of undelivered commodities when a van departs from a transshipment warehouse.

$$\sum_{j \in N_c \cup N_s} x_{i,j}^v \geq r_{ci} * o_{cr}^v, \forall v \in V, i \in N_s, r \in R, c \in C \quad (28)$$

$$\sum_{j \in N_c \cup N_s} x_{i,j}^v = \tilde{u}_v, \forall v \in V \quad (29)$$

$$\sum_{j \in N_c} x_{j,i}^v = \tilde{u}_v, \forall v \in V, i \in N_s \quad (30)$$

$$\sum_{j \in N_c \cup N_s} x_{i,j}^v = \sum_{j \in N_c \cup N_s} x_{j,i}^v, \forall v \in V, i \in N_c \quad (31)$$

$$\sum_{j \in N_c \cup N_s} \sum_{v \in V} g_{i,j}^{vd} + \sum_{j \in N_c \cup N_s} v_{i,j}^d = \tilde{u}_d \forall d \in D, \forall i \in N_c \cup N_s \quad (32)$$

$$\sum_{i \in N_c \cup N_s} \sum_{v \in V} g_{i,j}^{vd} + \sum_{i \in N_c \cup N_s} v_{i,j}^d = \sum_{i \in N_c \cup N_s} \sum_{v \in V} g_{j,i}^{vd} + \sum_{i \in N_c \cup N_s} v_{j,i}^d, \forall d \in D, j \in N_c \cup N_s \quad (33)$$

$$\sum_{j \in N_c \cup N_s} \sum_{i \in N_c \cup N_s} x_{i,j}^v * \tilde{d}_{ij} \leq \tilde{k}_v * \tilde{u}_v, \forall v \in V \quad (34)$$

$$\sum_{j \in N_c \cup N_s} \sum_{i \in N_c \cup N_s} v_{i,j}^d * \tilde{d}_{ij} \leq \tilde{k}_d * \tilde{u}_d, \forall d \in D \quad (35)$$

$$g_{i,j}^{vd} \leq x_{i,j}^v, \forall (i,j) \in A_2, v \in V, d \in D \quad (36)$$

$$\sum_{i \in N_c \cup N_s} x_{i,j}^v + \sum_{i \in N_c \cup N_s} v_{i,j}^d + \sum_{h \in N_c \cup N_s} x_{j,h}^v + \sum_{h \in N_c \cup N_s} v_{j,h}^d \geq 3k_j^{vd}, \forall j \in N_c, v \in V, d \in D \quad (37)$$

$$\sum_{i \in N_c \cup N_s} x_{i,j}^v + \sum_{i \in N_c \cup N_s} v_{i,j}^d + \sum_{h \in N_c \cup N_s} x_{j,h}^v + \sum_{h \in N_c \cup N_s} v_{j,h}^d \leq 2 + 2k_j^{vd}, \forall j \in N_c, v \in V, d \in D \quad (38)$$

Constraint (28) ensures that each van carrying requests can only depart from its corresponding transshipment warehouse. Constraints (29) and (30) specify that each van route starts from a transshipment warehouse and ends at a transshipment warehouse. Open routes are allowed, meaning that the ending transshipment warehouse may differ from the starting one. Constraint (31) guarantees the flow conservation of van routes. Constraint (32) ensures that each drone route starts from a transshipment warehouse and eventually returns to a transshipment warehouse. Similarly, open routes are allowed. Constraint (33) provides the flow conservation condition for drones. Constraints (34) and (35) impose maximum travel distance limits on both vans and drones. Constraints (36) illustrate the relation of $g_{i,j}^{vd}$ and $x_{i,j}^v$. Constraints (37) and (38) ensure that drones and vans synchronize at rendezvous nodes.

$$\sum_{i \in N_c \cup N_s} \sum_{v \in V} g_{i,j}^{vd} + \sum_{i \in N_c \cup N_s} v_{i,j}^d \leq 1, \forall j \in N_c \cup N_s, d \in D \quad (39)$$

$$\sum_{i \in N_c \cup N_s} \sum_{v \in V} x_{i,j}^v \leq 1, \forall j \in N_c \quad (40)$$

$$\tilde{h}_j \leq \sum_{d \in D} \sum_{i \in N_c \cup N_s} v_{i,j}^d, \forall j \in N_c \quad (41)$$

$$\sum_{v \in V} \sum_{i \in N_c \cup N_s} x_{i,j}^v \leq 1 - \tilde{h}_j, \forall j \in N_c \quad (42)$$

$$\sum_{i \in N_c \cup N_s} \sum_{v \in V} x_{i,j}^v + \sum_{d \in D} \sum_{i \in N_c \cup N_s} v_{i,j}^d \leq 1 + \sum_{d \in D} \sum_{v \in V} k_j^{vd}, \forall j \in N_c \quad (43)$$

$$\sum_{i \in N_c \cup N_s} \sum_{v \in V} x_{i,j}^v + \sum_{d \in D} \sum_{i \in N_c \cup N_s} v_{i,j}^d \geq 1 - \sum_{d \in D} \sum_{v \in V} k_j^{vd}, \forall j \in N_c \quad (44)$$

Constraints (39) restrict each drone to visit each customer at most once. Constraints (40) guarantee that each van can access each customer node at most once. Constraints (41) and (42) specify that requests destined for high-floor customers must be served by drones, while low-floor customers can be served either by vans or by drones. Constraints (43) and (44) ensure that each customer is served exactly once, either by a van or by a drone, except in cases where drone launching or retrieval operations occur at the same node.

$$a_j^v \geq a_i^v + \tau_{ij}^v - M(1 - x_{i,j}^v), \forall i, j \in N_c \cup N_s, v \in V \quad (45)$$

$$a_j^v \geq a_i^d + \tau_{ij}^v - M(1 - x_{i,j}^v), \forall i, j \in N_c \cup N_s, v \in V, d \in D \quad (46)$$

$$d_j^v \geq a_j^v, \forall v \in V, j \in N_c \cup N_s \quad (47)$$

$$a_j^v \geq d_i^v + \tau_{i,j}^v - M(1 - x_{i,j}^v), \forall i, j \in N_c \cup N_s, v \in V \quad (48)$$

$$a_j^d \geq a_i^d + \tau_{i,j}^d - M(1 - v_{i,j}^d), \forall i, j \in N_c \cup N_s, d \in D \quad (49)$$

$$d_j^d \geq a_j^d, \forall d \in D, j \in N_c \cup N_s \quad (50)$$

$$a_j^d \geq a_i^v + \tau_{i,j}^d - M(1 - v_{i,j}^d), \forall i, j \in N_c \cup N_s, v \in V, d \in D \quad (51)$$

$$a_j^d \geq d_i^d + \tau_{i,j}^d - M(1 - v_{i,j}^d), \forall i, j \in N_c \cup N_s, d \in D \quad (52)$$

$$l_i + M(1 - \sum_{j \in N_c \cup N_s} x_{i,j}^v) \geq a_i^v \geq e_i - M(1 - \sum_{j \in N_c \cup N_s} x_{i,j}^v), \forall i \in N_c, v \in V \quad (53)$$

$$l_i + M(1 - \sum_{j \in N_c \cup N_s} v_{i,j}^d) \geq a_i^d \geq e_i - M(1 - \sum_{j \in N_c \cup N_s} v_{i,j}^d), \forall i \in N_c, d \in D \quad (54)$$

Constraints (45) - (48) describe the arrival time update relationship for a van traveling from node i to j . Constraints (49) - (52) similarly define the time propagation relationship for a drone flying from node i to j . Specifically, constraints (46) and (51) enforce the temporal synchronization at rendezvous nodes by establishing mutual precedence relationships, which implicitly allow either the van or the drone to safely wait for the other to arrive. Constraints (53) and (54) ensure that the service time at each customer satisfies the corresponding time window constraints.

$$e_{id}^+ \leq M(1 - \sum_{v \in V} k_i^{vd}), \forall i \in N_c \cup N_s, d \in D \quad (55)$$

$$e_{jd}^- \geq e_{id}^+ + [\beta_1(P_{id}^+ + D_{id}^+ + m) + \beta_0]\tau_{ij}^d - M(1 - v_{i,j}^d), \forall i, j \in N_c \cup N_s, d \in D \quad (56)$$

$$e_{id}^+ \geq e_{id}^- - M * \sum_{v \in V} k_i^{vd}, \forall i, j \in N_c \cup N_s, d \in D \quad (57)$$

$$e_{jd}^- \leq \tilde{e}_d * \sum_{i \in N_c \cup N_s} v_{i,j}^d, \forall j \in N_c \cup N_s, d \in D \quad (58)$$

Constraints (55) state that each drone is fully charged when it departs from a van and enters an independent state of flight. Constraints (56) and (57) update the cumulative energy consumption of a drone upon its arrival at node j . Constraints (58) guarantee that the cumulative energy consumption of each drone throughout its independent flight mission does not exceed its battery endurance limit.

$$D_{jv} \leq D_{iv} + M(1 - x_{i,j}^v), \forall j \in N_s \cup N_c, i \in N_s, v \in V \quad (59)$$

$$P_{jv} \geq P_{iv} - M(1 - x_{i,j}^v), \forall j \in N_s \cup N_c, i \in N_s, v \in V \quad (60)$$

$$D_{jv} \geq D_{iv} - d_i - \sum_{d \in D} D_{id}^+ - M(1 - x_{i,j}^v), \forall i \in N_c, j \in N_s \cup N_c, v \in V \quad (61)$$

$$D_{jv} \leq D_{iv} - d_i - \sum_{d \in D} D_{id}^+ + M(1 - x_{i,j}^v), \forall i \in N_c, j \in N_s \cup N_c, v \in V \quad (62)$$

$$P_{jv} \geq P_{iv} + p_i + \sum_{d \in D} P_{id}^- - M(1 - x_{i,j}^v), \forall i \in N_c, j \in N_s \cup N_c, v \in V \quad (63)$$

$$P_{jv} \leq P_{iv} + p_i + \sum_{d \in D} P_{id}^- + M(1 - x_{i,j}^v), \forall i \in N_c, j \in N_s \cup N_c, v \in V \quad (64)$$

$$P_{iv} + D_{iv} \leq \tilde{c}_v, \forall i \in N_s \cup N_c, v \in V \quad (65)$$

$$D_{id}^+ \leq D_{id}^- - d_i(1 - \sum_{v \in V} k_i^{vd}) + M(1 - v_{i,j}^d), \forall i \in N_c, j \in N_s \cup N_c, d \in D \quad (66)$$

$$D_{jd}^- \leq D_{id}^+ + M(1 - v_{i,j}^d), \forall i \in N_c, j \in N_s \cup N_c, d \in D \quad (67)$$

$$D_{jd}^- \geq D_{id}^+ - M(1 - v_{i,j}^d), \forall i \in N_c, j \in N_s \cup N_c, d \in D \quad (68)$$

$$P_{id}^+ \geq P_{id}^- + p_i(1 - \sum_{v \in V} k_i^{vd}) - M(1 - v_{i,j}^d), \forall i \in N_c, j \in N_s \cup N_c, d \in D \quad (69)$$

$$P_{jd}^- \leq P_{id}^+ + M(1 - v_{i,j}^d), \forall i \in N_s \cup N_c, j \in N_s \cup N_c, d \in D \quad (70)$$

$$P_{jd}^- \geq P_{id}^+ - M(1 - v_{i,j}^d), \forall i \in N_s \cup N_c, j \in N_s \cup N_c, d \in D \quad (71)$$

$$D_{id}^- \leq M(1 - \sum_{v \in V} k_i^{vd}), \forall i \in N_s \cup N_c, d \in D \quad (72)$$

$$P_{id}^+ \leq M(1 - \sum_{v \in V} k_i^{vd}), \forall i \in N_s \cup N_c, d \in D \quad (73)$$

$$P_{id}^- + D_{id}^- \leq \tilde{c}_d, \forall i \in N_s \cup N_c, d \in D \quad (74)$$

$$P_{id}^+ + D_{id}^+ \leq \tilde{c}_d, \forall i \in N_s \cup N_c, d \in D \quad (75)$$

$$x_{i,j}^v \in \{0,1\}, i, j \in N_+, v \in V \quad (76)$$

$$y_{i,j}^k \in \{0,1\}, i, j \in N_0, k \in K \quad (77)$$

$$u_{i,j}^{k,p} \in \{0,1\}, i, j \in N_0, k \in K, p \in P \quad (78)$$

$$v_{i,j}^d \in \{0,1\}, i, j \in N_+, d \in D \quad (79)$$

$$g_{i,j}^{vd} \in \{0,1\}, i, j \in N_+, d \in D, v \in V \quad (80)$$

$$k_i^{vd} \in \{0,1\}, i \in N_+, d \in D, v \in V \quad (81)$$

$$\tilde{u}_i \in \{0,1\}, i \in K \cup V \cup D \quad (82)$$

$$o_{cr}^v \in \{0,1\}, c \in C, r \in R, v \in V \quad (83)$$

$$z_{cp} \in \{0,1\}, c \in C, p \in P \quad (84)$$

$$r_{ci} \in \{0,1\}, c \in C, i \in N_s \quad (85)$$

$$m_{cv} \in \{0,1\}, c \in C, v \in V \quad (86)$$

$$h_c \geq 0, c \in C \quad (87)$$

$$a_i^k, d_i^k \geq 0, i \in N_0, k \in K \quad (88)$$

$$D_{iv}, P_{iv}, a_i^v, d_i^v \geq 0, i \in N_+, v \in V \quad (89)$$

$$D_{id}^-, D_{id}^+, P_{id}^-, P_{id}^+, a_i^d, d_i^d, e_{id}^-, e_{id}^+ \geq 0, i \in N_+, d \in D \quad (90)$$

Constraints (59) and (60) update the pickup and delivery loads of a van after it departs from a transshipment warehouse. When the van travels from node i to j , constraints (61) - (64) update its pickup

load and delivery load, respectively. Constraints (65) ensure that the total load carried by each van does not exceed its maximum capacity. Constraints (66) - (71) describe the cumulative load updates of drones during delivery and pickup operations, respectively. Constraints (72) and (73) specify that the cumulative delivered and picked-up loads of a drone are reset at launch or retrieval nodes. Constraints (74) and (75) ensure the payload of the drone always respects its maximum capacity throughout the entire operation. Constraints (76) - (90) specify the variable domains.

The container drayage for export requests must be conducted strictly after the completion of the pickup and delivery operations. Otherwise, it is impossible to determine the precise arrival times of the collected commodities at the transshipment warehouses, as these times depend on the coupled scheduling of the van-drone fleet.

In Stage 1, the model determines the routing and scheduling of the trucks and van-drone fleet, and the resulting arrival times and locations of commodities at the transshipment warehouses. The first-stage model inherently dictates the final locations and the arrival times of the export requests at the warehouses. These values are extracted as fixed parameters and passed forward to initialize the second stage, thereby guaranteeing spatiotemporal feasibility between the stages.

In Stage 2, the model optimizes the routing and scheduling of trucks transporting containers from transshipment warehouses to ports, subject to the availability constraints inherited from Stage 1. This sequential structure ensures that second-stage operations remain consistent with the completion of first-stage pickup and delivery activities, thereby maintaining spatiotemporal feasibility across stages. The second-stage model shares a similar structure with the container drayage component of the first stage, where a tractor hauling a trailer transports an export container from a transshipment warehouse to the port. The primary distinction is that the destinations $\tilde{d}_c$ of all containers in the second stage are designated as the port nodes. To avoid redundancy, the mixed-integer linear programming formulation for the second stage is provided in Appendix A. To further clarify the information flow between the two stages, two columns have been added to Table 2 to specify their stage-wise involvement. A checkmark in these columns indicates that the decision variables are involved in the corresponding stage.

5. Solution approach

Based on the descriptions elaborated in Section 3, after transporting import containers via the first phase,

the deliveries for the import requests and the pickups for the export requests are completed simultaneously during the second phase, while the container drayage for the export requests must be conducted after the completion of the second phase because otherwise it is impossible to know the arrival times of the export requests at the warehouses. Namely, the arrival times at warehouses for export requests depend on the scheduling of serving the import requests. Therefore, we utilize a two-stage solution approach to solve the TVDCTP-T problem, where the first stage focuses on servicing import requests and collecting the commodities for export requests to warehouses simultaneously. By utilizing the generated arrival times at the warehouses for export requests, the second stage solves a container drayage problem to transport the export commodities to the ports. The formulation for the first stage is presented in Section 4, and we move the formulation for the second stage to Appendix A. because it is the same as the container drayage part of the formulation in the first stage.

The TVDCTP-T problem involves multi-stage transportation decisions and strong spatiotemporal coupling among heterogeneous carriers under capacity, time-window, and drone energy constraints. The resulting solution space is highly combinatorial and tightly coupled, making exact optimization methods computationally intractable for medium-scale and large-scale instances. To address this challenge, we propose a reinforcement learning-enhanced cascade-based Adaptive Large Neighborhood Search algorithm, termed as RL-C-ALNS. Specifically, the component C-ALNS extends the conventional ALNS framework by explicitly modeling the structural dependencies between van routes and drone sub routes. By incorporating cascade-aware destroy and repair mechanisms, local modifications to customer assignments can trigger synchronized updates across truck, van, and drone operations, as the impact propagates from a removed or modified customer node to all structurally dependent nodes through multi-stage coordination relationships, forming a unified structure of affected routes and sub-routes that must be jointly updated. This enables effective exploration of a larger structural neighborhood. Based on this mechanism, two cascade-aware operators are developed in this study, namely Cascade Removal and Multi-node Cascade Repair, which perform coordinated destruction and reconstruction over these structurally dependent customer sets.

The RL component is adopted to facilitate operator selection, where the reinforcement learning agent takes over the selection of destroy-repair operator pairs within the ALNS framework, which can further enhance the adaptivity across the entire search process to better deal with the complex and nonstationary characteristics between search states and effective operators. In the RL-C-ALNS procedure, the agent is

solely responsible for operator selection, while all other components, including neighborhood structures, solution evaluation, and the simulated annealing acceptance mechanism, follow the standard C-ALNS process.

Figure 3 illustrates the overall framework of the proposed RL-C-ALNS algorithm. First, an initial solution S_0 is constructed using a greedy heuristic. At iteration t , the current solution S_t is represented by its corresponding state s_t , which is fed into the actor network. Based on s_t , the actor outputs an action a_t , which corresponds to selecting a pair of destroy and repair operators. The selected operator pair is then applied to the current solution S_t to generate a candidate solution S_{t+1} . The candidate solution is evaluated by the simulated annealing (SA) acceptance criterion together with the adaptive mechanism. If the acceptance condition is satisfied, the current solution is updated as $S_t \leftarrow S_{t+1}$. Meanwhile, the global best solution S_{MAX}^* is updated whenever an improved solution is found. The reward r_t is computed based on the acceptance outcome and the objective improvement, and the transition is stored in the experience buffer. The iteration counter is then increased ($t \leftarrow t + 1$), and the process continues until the maximum number of iterations N_{max} is reached.

In Figure 3, for PPO, at each policy update, a mini batch is sampled from the experience buffer and passed to the critic network to produce value estimates and compute the advantages $\bar{A}_t$. The actor then updates the policy by combining the computed advantages with the parameters of the old policy, yielding the updated network parameters for the new policy.

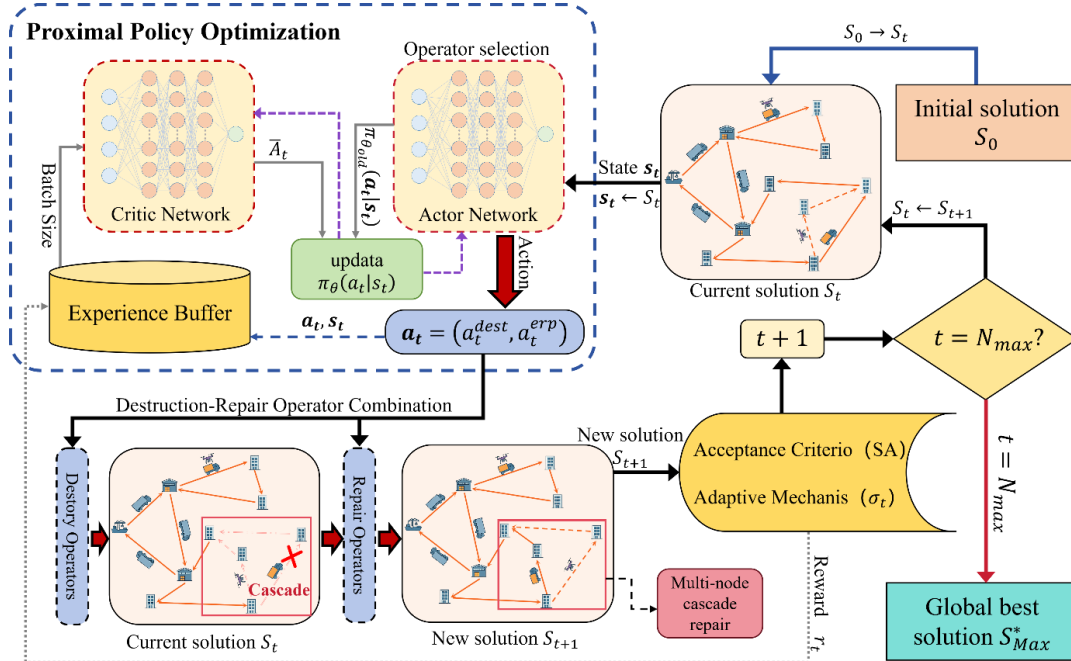


Figure 3 Overall architecture of the RL-C-ALNS. The algorithm starts with an initial solution constructed by a greedy

heuristic. At each iteration, the current solution is encoded as a state and fed into the actor network, which selects a destroy-repair operator pair. The selected operators are applied to generate a candidate solution, which is evaluated using the simulated annealing (SA) acceptance criterion together with an adaptive mechanism. The current solution is updated based on the acceptance decision, while the best global solution is tracked throughout the search process. The reward signal, derived from the acceptance outcome and objective improvement, is stored in the experience buffer and used to update the actor-critic networks via PPO.

5.1. Cascade-based ALNS for Truck-Van-Drone collaboration

The overall procedure of C-ALNS follows the standard ALNS paradigm and can be summarized as follows, where each component is detailed in the subsections.

- **Initial solution:** an initial feasible solution is constructed using a greedy heuristic that explicitly incorporates truck-van-drone coordination and serves as the starting point of the search.
- **Iterative refinement process:** starting from the initial solution, C-ALNS performs an iterative destroy-repair search for a predefined number of iterations to progressively improve solution quality.
- **Adaptive operator selection:** in each iteration, destroy and repair operators are adaptively selected based on their historical performance to balance exploration and exploitation.
- **Candidate solution generation:** the selected destroy and repair operators are applied with cascade mechanisms to generate a structurally consistent candidate solution.
- **Evaluation and acceptance:** each candidate solution is evaluated using the objective function and accepted according to a simulated annealing criterion, while the global best solution is updated whenever an improvement is found.
- **Stopping criterion:** the iterative process terminates when the maximum number of iterations (or a time limit) is reached, and the best solution obtained is returned.

5.1.1. Initial Solution

To ensure that the search starts from a feasible and structurally consistent solution, we adopt a greedy heuristic to construct the initial solution, which explicitly reflects the hierarchical structure of the truck-van-drone system. The initial solution is generated in a staged manner, which also provides a stable foundation for subsequent cascade-based destroy and repair operations. The construction procedure consists of three stages.

Stage 1: Container-tractor transportation. Each container is assigned to a transportation path operated by a tractor-trailer combination, delivering the container from its origin (port or intermediate depot) to a

designated hub. This stage guarantees that all containers are properly scheduled and that container availability at hubs is ensured before last-mile operations.

Stage 2: Van main-route construction. At the hub level, van routes are constructed using a greedy insertion strategy. Each van starts and ends at a hub, and customer nodes are inserted sequentially while checking vehicle capacity and time window feasibility. If no existing van can accommodate a new customer, an additional van is activated.

Stage 3: Drone-assisted refinement. After van routes are established, part of the customer demand is shifted from vans to drones whenever feasible. For customers satisfying drone payload, endurance, and time window constraints, drone sub routes of the form (launch-service-recovery) are constructed. This step reduces van travel distance and explicitly establishes van-drone coordination structures.

Compared with random initialization, this staged construction method significantly reduces the number of infeasible initial solutions and ensures that cooperative structures between vans and drones are explicitly represented from the beginning of the search, which is essential for the effectiveness of cascade-based neighborhood operators.

5.1.2. Destroy Operators

In each iteration, a destroy operator removes a subset of customer nodes from the current solution and places them into an unserved set. Due to the strong structural coupling between van routes and drone sub routes, the removal of a single customer node may invalidate multiple dependent drone operations. Therefore, destroy operators are designed to operate not only at the node level but also at the coordination-structure level.

Let S denote the current solution and N_c denote the set of customer nodes currently served by van routes, respectively. The designed destroy operators are elaborated as follows:

- Random customer removal: A predefined proportion of customer nodes is randomly selected from van routes and removed, enhancing diversification of the search process.
- Greedy removal: For each customer $i \in N_c$, its marginal contribution to the objective function is defined as $\Delta f_i = f(S) - f(S/i)$. Customers are ranked in descending order of Δf_i , and the top k customers are selected:

$$\mathcal{R} = \underset{\mathcal{R} \subseteq N_c}{\operatorname{argmax}} \mid \mathcal{R} \mid = k \sum_{i \in \mathcal{R}} \Delta f_i \quad (91)$$

- Related removal: A seed customer $i \in N_c$ is selected uniformly at random. The relatedness

between customers i and j is measured by their spatial distance $\phi_{ij} = d_{ij}$. Customers are sorted in ascending order of ϕ_{ij} , and the first k customers form the removal set:

$$\mathcal{R} = \{(i, j) \in N_c \mid \phi_{ij} \leq \phi_k\} \quad (92)$$

- Cascade removal: It propagates the removal from a selected customer node to all structurally dependent nodes across truck, van, and drone operations, reflecting the multi-stage interdependencies induced by coordination decisions.

Let $\mathcal{R}^{(0)} \subset N_c$ denote the initial set of customers selected by any of the above removal strategies. Define $\mathcal{D}_i$ as the set of customer nodes that are structurally dependent on customer i through truck-van-drone coordination relationships, including those contained in the same drone sub-routes or affected by associated truck and van route decisions. The cascade removal set is recursively expanded as:

$$\mathcal{R}^{(t+1)} = \mathcal{R}^{(t)} \cup \mathcal{D}_i, i \in \mathcal{R}^{(t)} \quad (93)$$

until the following termination condition is satisfied $\mathcal{R}^{(t+1)} = \mathcal{R}^{(t)}$. The final cascade removal set is given by $\mathcal{R}^* = \mathcal{R}^{(t)}$. All customers in $\mathcal{R}^*$, together with their associated drone sub-routes, are removed simultaneously from the solution.

It is important to note that cascade removal is not a simple multi-node deletion strategy. Instead, it operates at the structural level by explicitly exploiting dependency relationships across truck, van, and drone operations. For example, a change in a truck-level decision, such as the selection of an intermediate warehouse, may invalidate the associated truck route as well as all downstream van and drone operations. Similarly, under flexible drone launch and recovery across vehicles, a removed node may affect an entire drone sortie and the subsequent van route that receives the drone, requiring these interconnected routes to be removed or updated simultaneously. This mechanism enables large-scale, coordination-aware perturbations of the solution.

5.1.3. Repair Operators

Repair operators reinsert customers from the unserved set into the current solution. Each insertion may influence the feasibility and quality of subsequent insertions, requiring careful evaluation. The proposed repair operators are detailed as follows:

- Global greedy repair: For each customer $i \in N_c$, all feasible insertion positions on existing van routes and feasible drone sub routes are evaluated. The optimal insertion strategy is selected by

$$\pi^*(i) = \underset{\pi \in \Omega(i)}{\operatorname{argmin}} f(S \oplus_{\pi} i) \quad (94)$$

where $\Omega(i)$ denotes the set of all feasible insertion strategies for customer i , and $S \oplus_{\pi} i$ represents the solution obtained by inserting customer i using strategy π .

- Local greedy repair: For each customer, a single van route is selected, and the customer is inserted at the best feasible position within that route $\pi^*(i)$.
- Regret-based repair: For each customer, let $f_1(i)$ and $f_2(i)$ denote the smallest and second smallest objective values obtained from feasible insertion strategies. The regret value is defined as $\text{Regret}(i) = f_2(i) - f_1(i)$. At each iteration, the customer with the largest regret value is selected and inserted using its best strategy.
- Multi-node cascade repair: It reconstructs and reinserts groups of structurally dependent customer nodes across truck, van, and drone operations.

Let $\mathcal{B} \subseteq \mathcal{R}^{(t)}$ denote a bundle of customers selected for joint insertion, where each bundle is formed from customer sets removed together during the cascade removal process.

$$\pi^*(\mathcal{B}) = \underset{\pi \in \Omega(\mathcal{B})}{\operatorname{argmin}} f(S \oplus_{\pi} \mathcal{B}) \quad (95)$$

subject to drone payload and endurance constraints, van route feasibility, truck-van consistency, van-drone coordination consistency, and time-window constraints.

Multi-node cascade repair does not insert customers independently but restores interdependent structures coordinately. For truck-van coupling, truck-level changes (e.g., intermediate warehouse reassignment) require corresponding van route reconstruction. For van-drone coupling, repairing a disrupted drone sortie may need simultaneous adjustment of launch and receiving van routes to maintain feasibility.

5.1.4. Acceptance Criterion and Adaptive Mechanism

The proposed C-ALNS adopts a simulated annealing (SA) acceptance criterion. In the early stages of the search, worse solutions may be accepted with a certain probability, which helps the algorithm escape local optima under complex constraints and strong structural coupling. As the temperature decreases, the search gradually shifts toward exploitation.

Let S denote the current solution and S' the candidate solution generated by the destroy-repair process. The acceptance probability is defined as:

$$P(S_t \rightarrow S') = \begin{cases} 1 & , f(S') < f(S_t) \\ \exp\left(-\frac{f(S')-f(S_t)}{T_t}\right) & , \text{otherwise} \end{cases} \quad (96)$$

where T is the current temperature. The temperature is updated according to a geometric cooling schedule:

$T_{t+1} = \alpha T_t$, where $\alpha \in (0,1)$ is the cooling coefficient.

Adaptive operator selection in C-ALNS is realized through a score-based weight update mechanism following the adaptive ALNS framework of Ropke and Pisinger (2006). At the end of each fixed-length iteration segment, destroy-repair operator pairs are evaluated based on their contributions to solution improvement, and different scores are assigned according to the acceptance outcomes of the generated solutions. Operators that lead to better solutions receive higher scores and are assigned larger selection weights, thereby increasing their probability of being selected in subsequent iterations. After each iteration, the destroy-repair operator pair used in that iteration is assigned a score σ according to the solution outcome:

$$\sigma = \begin{cases} 5, & f(S'_t) < f(S^*) \\ 3, & f(S'_t) < f(S_t) \\ 1, & \text{accepted} \\ 0, & \text{otherwise} \end{cases} \quad (97)$$

5.1.5. Cascade Mechanism

To enhance the readability and reproducibility of the proposed cascade mechanism, Algorithm 1 provides a unified description of the cascade removal and cascade repair procedures. Unlike conventional ALNS approaches that operate on independent customer nodes, this study introduces structural dependency relationships to capture the coupling among nodes in the truck-van-drone collaborative transportation system.

In the cascade removal phase, an initial removal set is first generated by a base destroy operator. This set is then recursively expanded according to structural dependency relationships among nodes, such that all related customer nodes, along with their associated drone sub routes and vehicle route segments, are removed simultaneously. This process ensures that local modifications do not introduce inconsistencies into the coordination structure.

In the cascade repair phase, the removed customer nodes are partitioned into multiple bundles based on their dependency relationships, and each bundle is reconstructed jointly. For each bundle, the algorithm constructs a set of feasible insertion strategies under constraints including drone payload and endurance, vehicle route feasibility, truck-van consistency, van-drone coordination consistency, and time windows. The optimal insertion strategy is then selected to reinsert the bundle into the solution while restoring the associated coordination structure.

Overall, the proposed cascade mechanism consists of two components: dependency-aware recursive removal and bundle-based coordinated repair. Both stages explicitly account for the multi-level coordination relationships in the truck-van-drone system, thereby forming a structurally consistent neighborhood generation process.

Algorithm 1: Cascade Mechanism

Input: Current solution S , Initial removal set $R^{(0)}$

Output: Candidate solution S'

1. Initialize the cascade removal set $R \leftarrow R^{(0)}$
 2. **while** there exists a node $i \in R$ such that $\mathcal{D}_i \subseteq R$ **do**:
 3. Identify the structurally dependent node set $\mathcal{D}_i$ of node i based on cascade dependency relationships in the truck-van-drone system
 4. Update the removal set $R \leftarrow R \cup \mathcal{D}_i$
 5. **end while**
 6. Obtain the final cascade removal set R^*
 7. Remove all customer nodes in R^* from the current solution S , together with their associated drone sub routes, vehicle route segments, and coordination structures
 8. Partition R^* into a set of customer bundles $\mathcal{B}$ according to structural dependency relationships
 9. **for** each bundle $\mathcal{B}$ **do**:
 10. Construct the feasible insertion strategy set $\Omega(\mathcal{B})$
 11. Subject to drone payload and endurance constraints, vehicle route feasibility, truck-van consistency, van-drone coordination consistency, and time-window constraints
 12. Select the optimal insertion strategy:
 13. $\pi^*(\mathcal{B}) = \arg \min_{\pi \in \Omega(\mathcal{B})} f(S \oplus_{\pi} \mathcal{B})$
 14. Reinsert bundle $\mathcal{B}$ into the current solution and jointly reconstruct the associated vehicle routes, drone sub routes, and coordination relationships
 15. **end for**
 16. **Return** the repaired candidate solution S'
-

5.2. Reinforcement Learning-Enhanced C-ALNS (RL-C-ALNS)

The proposed C-ALNS adopts a heuristic weight-updating mechanism for operator selection. However, such handcrafted rules may be insufficient to capture complex and nonstationary relationships between search states and effective operators. To overcome this limitation, we introduce an RL component into the C-ALNS, i.e., the RL-C-ALNS, in which the heuristic operator selection mechanism is replaced by a reinforcement learning policy trained using the PPO algorithm. The incorporated RL component is utilized exclusively to select destroy-repair operator combinations. The underlying ALNS framework, neighborhood

structures, objective function, and acceptance criterion remain unchanged, ensuring that the interpretability and stability of the original heuristic approach are preserved while enabling more adaptive operator selection. The operator selection process in RL-C-ALNS is modeled as a Markov Decision Process (MDP) defined by the tuple $\langle \mathbf{S}, \mathbf{A}, \mathbf{R}, \mathbf{P} \rangle$, where $\mathbf{S}$ is the state matrix, $\mathbf{A}$ is the action matrix, $\mathbf{R}$ is the reward vector, and $\mathbf{P}$ is the policy, each element in the tuple is elaborated in the following sections.

5.2.1. State

The state $\mathbf{s}_t \in \mathbf{S}$ is represented by a composite feature vector formed by concatenating three components:

$$\mathbf{s}_t = [\mathbf{s}_t^{\text{update}}, \mathbf{s}_t^{\text{slack}}, \mathbf{s}_t^{\text{temporal}}] \quad (98)$$

where $\mathbf{s}_t^{\text{update}}$ characterizes the recent solution improvement behavior, $\mathbf{s}_t^{\text{slack}}$ reflects the temporal and capacity slack of the current solution, and $\mathbf{s}_t^{\text{temporal}}$ encodes the recent usage and performance history of destroy-repair operators. Together, these components provide the reinforcement learning agent with a compact yet informative representation of the current search state, enabling adaptive operator selection across different search stages.

Specifically, $\mathbf{s}_t^{\text{update}}$ characterizes the update status of the candidate solution relative to the current and global best solutions, and is defined as:

$$\mathbf{s}_t^{\text{update}} = (u_t, \Delta_t) \quad (99)$$

where u_t denotes the categorical update indicator and Δ_t represents the relative objective change between consecutive solutions. u_t is encoded as $u_t \in \{0, 1, 2, 3\}$, where $u_t = 0$: the candidate solution is rejected; $u_t = 1$: the candidate solution is accepted but does not improve the global best solution; $u_t = 2$: the candidate solution improves the global best solution but is not accepted; and $u_t = 3$: the candidate solution is accepted and improves the global best solution. The relative objective change Δ_t is defined as:

$$\Delta_t = \frac{f(S_t) - f(S'_t)}{f(S_t)} \quad (100)$$

where S_t and S'_t denote the current and candidate solutions, respectively.

$\mathbf{s}_t^{\text{slack}}$ captures the degree of temporal and capacity tightness of the current solution, and is defined as:

$$\mathbf{s}_t^{\text{slack}} = [t_t^{\min}, t_t^{\text{avg}}, l_t^{\max}, l_t^{\text{avg}}] \quad (101)$$

where $t_t^{\min}$ and t_t^{avg} denote the minimum and average time-window slack over all serviced nodes, respectively, while $l_t^{\max}$ and l_t^{avg} represent the minimum and average residual capacity slack of the vehicles along their routes.

s_t^{temporal} summarizes the recent usage and performance history of destroy and repair operators, and is defined as:

$$s_t^{\text{temporal}} = (\mathcal{f}_t^{\text{usage}}, \mathcal{s}_t^{\text{score}}, o_t^{\text{recent}}) \quad (102)$$

where $\mathcal{f}_t^{\text{usage}}$ and $\mathcal{s}_t^{\text{score}}$ denote the normalized usage frequencies and average historical scores of each operator within a fixed-length sliding window, respectively, and o_t^{recent} records the sequence of recently selected destroy-repair operator pairs.

5.2.2. Action

The action space $\mathbf{A}$ consists of all combinations of four destroy operators and four repair operators, resulting in 16 candidate operator pairs. An action $\mathbf{a}_t \in \mathbf{A}$ corresponds to an operator selection decision in a single heuristic iteration t and is defined as a combination of a destroy operator and a repair operator, shown as follows:

$$\mathbf{a}_t = (a_t^{\text{dest}}, a_t^{\text{rep}}) \quad (103)$$

where a_t^{dest} denotes the destroy operator used to remove a subset of customers or nodes from the current solution, and a_t^{rep} denotes the repair operator responsible for reinserting the removed elements. For the TVDCTP-T problem studied in this paper, cascade mechanisms are embedded in destroy operators to explicitly model structural dependencies among tasks, while repair operators combine multiple heuristic insertion strategies to ensure feasibility and solution quality.

5.2.3. Reward

The reward function evaluates the effectiveness of an operator selection decision by combining acceptance-based feedback and objective improvement information, and generates a reward $r_t \in \mathbf{R}$ where r_t is defined as:

$$r_t = \alpha_{\text{reward}} \sigma_t + \Delta_t \quad (104)$$

where σ_t denotes the score assigned to the selected destroy-repair operator pair in the current heuristic iteration according to the acceptance-based scoring mechanism introduced in Section 5.1.4, and α_{reward} is a weighting parameter ranging between $[0, 1]$. Specifically, α_{reward} controls the influence of acceptance-based feedback relative to objective improvement. By integrating these two components, the reward formulation encourages operator selections that are both compatible with the acceptance criterion and effective in improving solution quality across different search stages.

5.2.4. State Transition and Proximal Policy Optimization (PPO)

In the proposed RL-C-ALNS algorithm, each heuristic iteration is treated as a single decision step for reinforcement learning. At state $\mathbf{s}_t$, the agent selects an action $\mathbf{a}_t$, corresponding to a destroy-repair operator pair. The environment then executes the selected operators, applies the simulated annealing acceptance criterion to update the current solution, and returns the reward r_t along with the next state $\mathbf{s}_{t+1}$.

Based on the MDP formulation, the Proximal Policy Optimization (PPO) algorithm is adopted to learn the operator selection policy $\mathbf{P}$ and replace the heuristic-based operator weight update mechanism in traditional ALNS. PPO belongs to the family of policy gradient methods and directly optimizes a parameterized stochastic policy. It is particularly well suited for the RL-ALNS setting due to its training stability under discrete action spaces and its robustness in nonstationary and iterative search environment. Through continuous interaction with the search process, PPO learns to identify more suitable operator selection strategies for different search stages.

The core idea of PPO is to restrict the magnitude of policy updates between successive iterations to ensure stable learning. Let the probability ratio between the new policy π_θ and the old policy $\pi_{\theta_{\text{old}}}$ for action $\mathbf{a}_t$ at state $\mathbf{s}_t$ be defined as:

$$\rho_t(\theta) = \frac{\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)}{\pi_{\theta_{\text{old}}}(\mathbf{a}_t|\mathbf{s}_t)} \quad (105)$$

Based on the advantage function $\bar{A}_t$, PPO employs the clipped surrogate objective as follows:

$$L^{\text{clip}}(\theta) = \mathbb{E}_t[\min(\rho_t(\theta)\bar{A}_t, \text{clip}(\rho_t(\theta), 1 - \epsilon, 1 + \epsilon)\bar{A}_t)] \quad (106)$$

where $\mathbb{E}_t[\cdot]$ denotes the empirical expectation over sampled decision steps, ϵ is the clipping threshold, and $\text{clip}(\cdot)$ truncates the ratio $\rho_t(\theta)$ to the interval $[1 - \epsilon, 1 + \epsilon]$ to prevent excessively large policy updates. The advantage estimate $\bar{A}_t$ is computed from the reward signal and the value function approximation, measuring how much better the selected operator pair $\mathbf{a}_t$ performs compared with the baseline value prediction at state $\mathbf{s}_t$. Since the reward integrates acceptance-based feedback and objective improvement information, maximizing $L^{\text{clip}}(\theta)$ directly encourages operator selections that yield higher overall search effectiveness while maintaining stable learning dynamics.

6. Numerical Experiments

In this section, we utilize both random-generated and real-world instances for numerical experiments. Three benchmark solution approaches are tested to demonstrate the performance of the proposed RL-ALNS

approach. Sensitivity analysis is conducted to provide managerial insights.

6.1. Experimental setup

To comprehensively evaluate the effectiveness and robustness of the proposed approach, both randomly generated instances and real-world Shenzhen instances are considered in the computational experiments. The random instances are designed to provide a controlled testing environment, in which problem characteristics such as customer distribution, demand levels, and time-window tightness can be systematically varied. This enables a clear assessment of the algorithm's scalability and sensitivity to key parameters under different problem sizes.

In contrast, the real-world Shenzhen instances incorporate realistic spatial structures, heterogeneous travel distances, and operational cost settings derived from actual urban logistics environments. By embedding real geographic coordinates, road-network-based van distances, and aerial drone flight distances, the Shenzhen cases capture practical constraints and spatial irregularities that cannot be fully represented by purely synthetic data. As a result, they serve as a rigorous testbed for evaluating the practical applicability of the proposed solution approach in realistic large-scale terminal transportation scenarios.

The instances are classified into three categories: small-, medium-, and large-scale instances. The small-scale instances involve 5, 10, and 15 customers; the medium-scale instances include 25 and 50 customers; and the large-scale instances consist of 75 and 100 customers. For each size, 10 independent instances are generated to reduce the influence of randomness on the experimental results. The ratio of the three types of requests (pure import, pure export, and combined) is set to 1:1:1 for all instances.

6.1.1. Random instances

All customer nodes are uniformly distributed within a square service region with a side length of 5 km, and inter-node distances are calculated using Euclidean distance.

In the randomly generated instances, each van is equipped with two drones. The number of vans available at each transshipment warehouse varies with the problem scale and is set to 2, 3, and 4 for the small-, medium-, and large-scale instances, respectively. Both delivery and pickup demands of customers are randomly generated within the interval $[0, 20]$. Parcels whose weights exceed the maximum payload capacity of drones are restricted to being served exclusively by vans. To enhance the diversity of temporal constraints, approximately 10% of customer nodes are assigned relatively tight service time windows.

The key parameters for vehicles and drones are set by following commonly used values in the literature. The tractor has a fixed daily operating cost of USD 28.62 per vehicle (Xiao & Gao, 2024), an average travel speed of 40 km/h, and a unit travel cost of USD 0.79 per km (Jia et al., 2022). The van has a fixed daily operating cost of USD 28.62 per vehicle, an average travel speed of 40 km/h, a unit travel cost of USD 0.79 per km, and a maximum payload capacity of 500 kg (Jiang et al., 2024). Considering the differences between drones and vans in terms of mobility and operating costs, the drone has a fixed daily operating cost of USD 6.44 per vehicle (Xiao & Gao, 2024), a flight speed of 80 km/h, a unit flight cost of USD 0.07 per km (Carlsson & Song, 2018), and a maximum payload capacity of 30 kg (Dayarian et al., 2020); its self-weight is set to 5 kg (Jiang et al., 2025). The drone energy consumption model follows the payload-linear regression model proposed by Dorling et al. (2017), where power demand is determined jointly by the effective payload weight and hovering power, and the battery capacity is set to 13.8 kWh. The time penalty cost is set to USD 5 per hour. Overall, these settings are consistent with existing studies on van-drone coordinated delivery and are intended to reasonably reflect practical operating characteristics.

6.1.2. Real-world instances

The real-world Shenzhen instances are constructed based on practical urban logistics operations. The port node is centered on Yantian Port, while multiple representative transshipment warehouses within Shenzhen are selected as intermediate facilities. Customer nodes are generated using real geographic coordinates. The travel distances of ground vehicles (including trucks and vans) are extracted from the OpenStreetMap road network and computed using shortest-path distances. The corresponding travel times are then calculated based on these distances and predefined vehicle speeds, as specified in Table 3. Whereas the flight distances of drones are approximated by straight-line (Euclidean) distances between nodes, and the corresponding flight times are computed by dividing the Euclidean distances by the drone speed, which is also provided in Table 3. Figure 4 presents the locations of the port, warehouse, and customers of an example instance with 15 requests.

Key parameters in Shenzhen instances including vehicle travel speeds, transportation costs, fixed activation costs, and driver time costs, are calibrated based on publicly available reports released by authoritative industry and academic institutions. The fixed activation cost of drones is modeled to reflect the depreciation of drone equipment. It should also be noted that standardized formulations and publicly available data for drone-related fixed costs remain limited in the current literature. Therefore, this cost

parameter is set to represent a reasonable order of magnitude for drone deployment in urban logistics systems. Additional components, such as maintenance, licensing, and operational overhead, could be incorporated as more authoritative data sources become available, which remains a direction for further refinement in future studies. These sources consist of the 2025 Survey Report on Truck Drivers' Employment and Living Conditions published by Renmin University of China (IELS, 2025) and the 2024 Annual Report on the Commercial Vehicle Market released by the China Association of Automobile Manufacturers (CAAM, 2024). The detailed parameter settings are summarized in Table 3. This calibration approach ensures the practical realism of the experimental settings while mitigating potential biases introduced by individual sample variations.

In the Shenzhen instances, the problem sizes are set to 5, 15, 25, 50, 75, and 100 customer nodes, respectively. For each scale, ten independent instances are generated to evaluate the performance of the proposed algorithm under realistic scenarios of varying sizes.



Figure 4 The test area and the locations of ports, warehouses, and customers in Shenzhen for real-world instances: an example of instance with 15 customers. The locations of warehouses and the Yantian port are fixed regardless of instances.

Table 3 Configurations of Shenzhen instances.

Parameter	Description	Value
Vehicle speed	Travel speed of tractor (tractor unit)	30 km/h
	Travel speed of van	40 km/h

	Flight speed of drone (UAV)	80 km/h
Unit transportation cost	Unit transportation cost of tractor	1.93 CNY/km
	Unit transportation cost of van	0.875 CNY/km
	Unit transportation cost of drone	0.052 CNY/km
Fixed usage cost	Fixed daily cost of tractor	166 CNY/day
	Fixed daily cost of van	59 CNY/day
	Fixed daily cost of drone	169 CNY/day
Vehicle capacity	Maximum payload capacity of van	500 kg
	Maximum payload capacity of drone	30 kg
Drone energy	Battery capacity	13.8 kWh
	Energy consumption coefficient per unit distance	Following Eq. (3) in Dorling et al. (2017)
Time cost	Driver time penalty cost	38.9 CNY/hour

The experiments were implemented on a workstation with Intel i9-14900K processors, 64 GB of RAM, and a Nvidia GeForce RTX 4090 graphics. Gurobi Optimizer 10.0.1 is utilized, and the tested algorithms were coded in Python 3.10. The hyperparameters of the RL-C-ALNS are summarized in Appendix B.

6.2. Benchmarks

To demonstrate the effectiveness of the RL-C-ALNS approach, we include three benchmarks for comparison including mixed integer programming model, Greedy heuristics, and C-ALNS.

These benchmark methods are selected to evaluate the proposed approach from different perspectives. Gurobi is employed to obtain optimal solutions for small-scale instances, providing a reference for model correctness and solution quality. The greedy heuristic serves as a constructive baseline, which is also used to generate the initial solution for the ALNS framework and to demonstrate the improvement achieved by subsequent search procedures. The C-ALNS method is included as a direct comparison under the same problem setting and algorithmic framework, allowing the effectiveness of the reinforcement learning-based operator selection in RL-C-ALNS to be clearly assessed.

Although other solution approaches were also considered, the studied problem exhibits strong multi-

stage coupling between truck-van-drone operations. Therefore, this study focuses on ALNS-based methods to better exploit the proposed cascade mechanism and to isolate the contribution of the reinforcement learning component within a unified framework.

6.2.1. Mixed integer programming model (Gurobi)

We utilize a common solver Gurobi 10.0.1 with academic license to solve each stage of the TVDCTP-T model (shown in Section 4).

6.2.2. Greedy heuristic approach (Greedy)

The greedy heuristic corresponds to the initial solution construction procedure used in the ALNS framework and is applied here as a standalone baseline without subsequent iterative improvement.

6.2.3. Cascade-based adaptive large neighborhood search (C-ALNS)

The distinction between the C-ALNS and the RL-C-ALNS lies in the operator selection mechanism. C-ALNS adopts the conventional score-based approach, while RL-C-ALNS formulates an MDP to train a policy for operator selection.

6.3. Results on small-scale random instances

Table 4 presents the objective values and the CPU runtimes for each small-scale instance over all benchmarks. We calculate the gaps (GAP_1) between the results of the tested approach and Gurobi as follows:

$$GAP_1 = \frac{Value_{Heuristics} - Value_{Gurobi}}{Value_{Gurobi}} \times 100\% \quad (107)$$

where a negative gap indicates that the tested approach achieves a lower cost, i.e., a better solution than Gurobi.

For instances with 5 requests, we can see that Gurobi achieves optimal solutions within 93 seconds on average, while the greedy approach runs much faster but obtains solutions with 31% gaps on average. In addition, both C-ALNS and RL-C-ALNS achieve optimal solutions for 9 out of 10 instances, with one instance obtaining a gap of 0.08%, which can be considered near-optimal. By comparing C-ALNS with RL-C-ALNS, we can see that RL-C-ALNS performs little improvement over C-ALNS, which is natural because C-ALNS has achieved optimal for most of the instances, while for the instance that has a gap of 0.08%, a possible reason is that in such a small search space, different operator choices tend to lead to similar neighborhoods, so RL-based operator selection does not materially change the search trajectory and often

converges to the same best solution as C-ALNS. Moreover, with a very short time/iteration budget, the long-horizon benefit of RL cannot fully accumulate. Indeed, the effect of RL appears for instances with larger scales, which will be elaborated in the following sections. From a runtime perspective, the proposed heuristic approaches (RL-C-ALNS and C-ALNS) consume much less time than Gurobi, indicating the efficiency of the proposed approaches. We can see that RL-C-ALNS takes slightly less CPU time than C-ALNS on average while achieving the same solutions, indicating that although RL does not improve solution quality on small instances, it directs the search to avoid low-contribution neighborhoods to reach the same near-optimal solution as C-ALNS with fewer iterations.

For instances with 15 requests, we can see that Gurobi cannot obtain optimal solutions within 3600 seconds but feasible solutions. Compared to Gurobi, the proposed C-ALNS achieves better solutions with a GAP_1 of -2.03% using significantly less runtime of 1 second. Moreover, compared with C-ALNS, the proposed RL-C-ALNS can achieve an even better solution (a GAP_1 of -5.93%) using slightly less runtime on average, which proves that the RL ability acts for a larger search space. Based on the above findings and analysis, we may conclude that the proposed RL-C-ALNS and C-ALNS are both effective and efficient.

Table 4 Results on small-scale random instances.

Instances	Gurobi		Greedy			C-ALNS			RL-C-ALNS			
	Value	CPU (s)	Value	CPU (s)	GAP ₁ (%)	Value	CPU (s)	GAP ₁ (%)	Value	CPU (s)	GAP ₁ (%)	
5	1	365.25	24	383.56	0.001	5.01%	365.56	0.10	0.08%	365.56	0.09	0.08%
	2	211.05	58	342.17	0.001	62.13%	211.05	0.10	0.00%	211.05	0.08	0.00%
	3	233.93	37	385.44	0.001	64.77%	233.93	0.09	0.00%	233.93	0.09	0.00%
	4	203.16	2	334.35	0.001	64.57%	203.16	0.08	0.00%	203.16	0.06	0.00%
	5	329.38	3	334.36	0.001	1.51%	329.38	0.08	0.00%	329.38	0.06	0.00%
	6	302.77	224	354.46	0.001	17.07%	302.77	0.08	0.00%	302.77	0.06	0.00%
	7	224.97	485	377.13	0.001	67.64%	224.97	0.09	0.00%	224.97	0.09	0.00%
	8	322.97	40	348.39	0.001	7.87%	322.97	0.06	0.00%	322.97	0.06	0.00%
	9	340.08	30	385.26	0.001	13.29%	340.08	0.07	0.00%	340.08	0.06	0.00%
	10	338.36	27	359.45	0.001	6.23%	338.36	0.06	0.00%	338.36	0.06	0.00%
Avg.	287.19	93	360.46	0.001	31.01%	287.22	0.08	0.01%	287.22	0.07	0.01%	
15	1	549.36	3600	618.47	0.05	12.58%	588.31	1.08	7.09%	547.72	1.01	-0.30%
	2	532.80	3600	860.08	0.05	61.43%	566.97	1.03	6.41%	531.45	1.02	-0.25%
	3	670.70	3600	842.80	0.04	25.66%	610.67	1.39	-8.95%	600.37	1.21	-10.49%
	4	640.81	3600	895.13	0.05	39.69%	686.30	0.72	7.10%	612.87	0.74	-4.36%
	5	620.71	3600	759.31	0.05	22.33%	528.01	1.21	-14.93%	525.27	1.17	-15.38%
	6	662.92	3600	892.09	0.04	34.57%	707.65	0.96	6.75%	672.38	0.69	1.43%
	7	726.48	3600	856.71	0.05	17.93%	632.47	0.90	-12.94%	628.32	0.90	-13.51%
	8	644.90	3600	596.18	0.05	-7.55%	591.42	1.52	-8.29%	586.20	1.03	-9.10%
	9	575.52	3600	878.07	0.06	52.57%	567.13	0.71	-1.46%	547.80	0.63	-4.82%
	10	737.34	3600	845.83	0.04	14.71%	729.64	1.50	-1.04%	718.88	1.24	-2.50%
Avg.	636.15	3,600	804.47	0.05	27.39%	620.86	1.10	-2.03%	597.13	0.96	-5.93%	

6.4. Results on medium- and large-scale random instances

Table 5 and Table 6 presents the results for medium- and large-scale random instances, respectively. For these instances, Gurobi cannot obtain a solution within 3600 seconds, making GAP_1 unavailable, and highlighting the need for heuristic approaches. Therefore, we calculate GAP_2 and GAP_3 for performance evaluation as follows:

$$GAP_2 = \frac{Value_{C-ALNS} - Value_{Greedy}}{Value_{Greedy}} \times 100\% \quad (108)$$

$$GAP_3 = \frac{Value_{RL-C-ALNS} - Value_{C-ALNS}}{Value_{C-ALNS}} \times 100\% \quad (109)$$

where GAP_2 indicates the gap between greedy solutions and C-ALNS solutions, while GAP_3 demonstrates the gap between C-ALNS and RL-C-ALNS solutions. Similar to GAP_1 , a negative gap indicates a better solution.

For the medium-scale random instances, compared with the Greedy baseline, C-ALNS consistently produces much better solutions, achieving average improvements of -14.10% (size 25) and -11.02% (size 50) in terms of GAP_2 . Moreover, the proposed RL-C-ALNS further improves over C-ALNS. Specifically, RL-C-ALNS obtains additional average gains of -3.19% (size 25) and -3.39% (size 50) in terms of GAP_3 , while using even less runtime. RL-C-ALNS is slightly faster on average for size 25, reducing CPU time from 4.21 s to 3.87 s, and much faster for size 50, reducing runtime from 56.34 s to 44.53 s.

The above results indicate that the RL component provides effective guidance for operator selection in C-ALNS, helping the search allocate effort to more promising neighborhoods and reducing unproductive exploration. The advantage becomes more evident as the instance size increases from 25 to 50, where the search space expands and the baseline C-ALNS may spend more time exploring fewer effective moves. Consequently, RL-C-ALNS achieves a better quality-time trade-off on medium-scale instances: it not only improves solution quality over C-ALNS but also yields a noticeable runtime reduction, proving that RL guidance becomes increasingly beneficial when the search space is larger and more complex.

Table 5 Results on medium-scale random instances.

Instances	Gurobi		Greedy		GAP_1 (%)	C-ALNS			RL-C-ALNS		
	Value	CPU (s)	Value	CPU (s)		Value	CPU (s)	GAP_2 (%)	Value	CPU (s)	GAP_3 (%)
1	-	3600	1313.65	0.25	-	986.95	3.84	-24.87%	915.79	4.09	-7.21%
2	-	3600	1386.83	0.28	-	1135.55	3.55	-18.12%	1022.57	3.55	-9.95%
3	-	3600	1508.50	0.21	-	1453.01	5.30	-3.68%	1453.01	4.26	0.00%
4	-	3600	1487.53	0.29	-	1339.75	4.64	-9.93%	1340.08	3.35	0.02%
25	5	3600	1303.87	0.21	-	991.91	4.14	-23.93%	971.69	3.06	-2.04%
	6	3600	1157.86	0.21	-	1110.81	4.04	-4.06%	914.28	6.17	-17.69%
	7	3600	1203.43	0.26	-	1072.56	4.79	-10.87%	1072.56	3.33	0.00%
	8	3600	1076.31	0.26	-	960.28	5.86	-10.78%	951.91	5.75	-0.87%
	9	3600	1291.30	0.21	-	1021.15	3.78	-20.92%	1069.41	3.02	4.73%

10	-	3600	1449.16	0.22	-	1247.81	2.12	-13.89%	1247.81	2.10	0.00%	
Avg.	-	3600	1317.84	0.24	-	1131.98	4.21	-14.10%	1095.91	3.87	-3.19%	
50	1	-	3600	2890.65	1.93	-	2439.39	35.98	-15.61%	2439.38	25.15	0.00%
	2	-	3600	2672.05	2.22	-	2152.16	95.45	-19.46%	2139.73	93.89	-0.58%
	3	-	3600	2502.04	2.31	-	2274.91	90.78	-9.08%	2040.16	81.24	-10.32%
	4	-	3600	2391.38	2.04	-	2157.48	27.36	-9.78%	1951.02	30.97	-9.57%
	5	-	3600	2293.63	2.04	-	1960.56	69.75	-14.52%	1935.99	44.90	-1.25%
	6	-	3600	2388.33	2.23	-	2318.19	30.25	-2.94%	2318.19	22.39	0.00%
	7	-	3600	2362.69	2.17	-	2204.28	35.49	-6.70%	1976.41	36.19	-10.34%
	8	-	3600	2590.29	2.00	-	2427.68	44.25	-6.28%	2426.99	37.74	-0.03%
	9	-	3600	2132.28	2.07	-	1751.83	80.98	-17.84%	1751.83	46.52	0.00%
	10	-	3600	2142.27	2.05	-	1993.81	53.13	-6.93%	1965.32	26.35	-1.43%
Avg.	-	3600	2436.56	2.11	-	2168.03	56.34	-11.02%	2094.50	44.53	-3.39%	

1040

1041 For large-scale random instances, compared to Greedy, C-ALNS gains major reductions in objective
1042 value, with average $GAP_2 = -19.34\%$ for size 75 and -19.35% for size 100. More importantly, the proposed
1043 RL-C-ALNS improves further over C-ALNS on both sizes. For size 75, RL-C-ALNS achieves an additional
1044 average improvement of $GAP_3 = -6.42\%$, reducing the mean objective from 3780.52 to 3537.68, while also
1045 decreasing runtime from 617.24 s to 573.61 s. For size 100, RL-C-ALNS continues to outperform C-ALNS
1046 with average $GAP_3 = -3.93\%$ and a notable time saving (889.38 s to 720.89 s). By glancing across various
1047 instances, we can see the superiority of the RL-C-ALNS is not limited to small marginal gains: several cases
1048 show substantial quality improvements (e.g., a double-digit GAP_3 for instance 3 on size 75) and large runtime
1049 reductions (e.g., CPU drops for instances 2 and 7-9 on size-100), while only 1 out of 20 instances remain
1050 ties ($GAP_3 = 0$).

1051 The above results show that, at a large scale, performance depends on efficiently steering the search
1052 through an enormous neighborhood space. Standard C-ALNS, despite being adaptive, can still spend
1053 considerable time on low-return operator sequences when the search space becomes highly combinatorial.
1054 On the contrary, RL-C-ALNS uses a learned policy to prioritize more promising operator choices, which (i)
1055 accelerates progress toward high-quality search regions and (ii) reduces time spent on unproductive
1056 exploration. As a result, RL guidance improves the quality-time balance precisely where it matters most:
1057 when exact solvers time out and the search space is too large for purely rule-driven operator scheduling.
1058 Overall, the results demonstrate that RL-C-ALNS consistently delivers lower objective values than C-ALNS
1059 with equal or less computational time, reinforcing its advantage on challenging large-scale instances.

1060

Table 6 Results on large-scale random instances.

Instances		Gurobi		Greedy		C-ALNS			RL-C-ALNS			
		Value	CPU (s)	Value	CPU (s)	GAP ₁ (%)	Value	CPU (s)	GAP ₂ (%)	Value	CPU (s)	GAP ₃ (%)
75	1	-	3600	5042.49	9.39	-	4027.01	495.94	-20.14%	3911.05	481.15	-2.88%

	2	-	3600	4756.66	7.44	-	3721.30	447.29	-21.77%	3501.50	428.83	-5.91%
	3	-	3600	4737.30	8.26	-	4045.61	770.46	-14.60%	3281.13	691.99	-18.90%
	4	-	3600	5013.99	8.20	-	3507.00	571.54	-30.06%	3449.91	588.13	-1.63%
	5	-	3600	4633.69	9.65	-	3662.91	700.47	-20.95%	3482.12	690.80	-4.94%
	6	-	3600	4032.94	8.27	-	3399.94	791.30	-15.70%	3083.82	740.89	-9.30%
	7	-	3600	4789.49	9.86	-	3709.94	612.44	-22.54%	3592.36	600.14	-3.17%
	8	-	3600	4754.13	9.57	-	3757.41	547.08	-20.97%	3382.50	346.34	-9.98%
	9	-	3600	4242.76	9.19	-	4005.99	429.71	-5.58%	3964.57	435.62	-1.03%
	10	-	3600	4868.09	10.38	-	3968.11	806.13	-18.49%	3727.80	732.24	-6.06%
	Avg.	-	3600	4687.15	9.02	-	3780.52	617.24	-19.34%	3537.68	573.61	-6.42%
100	1	-	3600	5795.81	17.78	-	4348.77	829.10	-24.97%	4064.34	801.49	-6.54%
	2	-	3600	4804.32	18.62	-	3959.93	858.46	-17.58%	3571.07	332.80	-9.82%
	3	-	3600	4294.71	21.22	-	3710.78	501.87	-13.60%	3607.87	500.92	-2.77%
	4	-	3600	4223.50	23.59	-	3912.47	927.60	-7.36%	3731.90	975.17	-4.62%
	5	-	3600	4390.71	21.81	-	4080.63	738.97	-7.06%	4009.85	733.62	-1.73%
	6	-	3600	4301.09	22.26	-	3788.95	626.42	-11.91%	3788.95	574.28	0.00%
	7	-	3600	5795.81	20.58	-	4232.26	1119.00	-26.98%	4012.96	869.02	-5.18%
	8	-	3600	6588.44	22.45	-	3809.89	1119.24	-42.17%	3631.14	905.81	-4.69%
	9	-	3600	4084.16	24.53	-	3540.99	1471.22	-13.30%	3462.82	1013.28	-2.21%
	10	-	3600	4777.51	20.14	-	4180.59	701.96	-12.49%	4130.22	502.49	-1.20%
	Avg.	-	3600	4905.61	21.30	-	3956.53	889.38	-19.35%	3801.11	720.89	-3.93%

1061

1062 6.5. Results on real-world instances

1063 Table 7 presents the results on small-scale real-world instances. For small-scale real-world instances
1064 with size 5, Gurobi achieves optimal within less than 2.5 seconds, and the proposed RL-C-ALNS and C-
1065 ALNS both reach optimal using significantly less time (0.08 seconds for C-ALNS and 0.09 seconds for RL-
1066 C-ALNS). For instances with size 15, Gurobi cannot solve to optimality within 3600 seconds, while C-
1067 ALNS is able to generate solutions with less than 6% higher costs within less than 1 second. Compared with
1068 C-ALNS, the proposed RL-C-ALNS can further increase the solution quality by 3.8% with just 0.02 seconds
1069 more CPU time. The above findings indicate the effectiveness and efficiency of the proposed RL-C-ALNS
1070 in real-world cases.

1071 Table 7 Results on small-scale real-world instances.

Instances	Gurobi			Greedy			C-ALNS			RL-C-ALNS		
	Value	CPU (s)		Value	CPU (s)	GAP ₁ (%)	Value	CPU (s)	GAP ₁ (%)	Value	CPU (s)	GAP ₁ (%)
5	1	976.35	3.09	1015.07	0.001	3.97%	976.35	0.09	0.00%	976.35	0.09	0.00%
	2	1164.98	1.68	1659.37	0.001	42.44%	1164.98	0.09	0.00%	1164.98	0.09	0.00%
	3	1006.14	2.62	1249.76	0.001	24.21%	1006.14	0.12	0.00%	1006.14	0.12	0.00%
	4	946.53	1.38	1096.09	0.001	15.80%	946.53	0.06	0.00%	946.53	0.11	0.00%
	5	1183.27	1.45	1681.93	0.001	42.14%	1183.26	0.06	0.00%	1183.26	0.05	0.00%
	6	1176.88	1.31	1623.74	0.001	37.97%	1176.88	0.09	0.00%	1176.88	0.09	0.00%
	7	1164.52	1.12	1816.30	0.001	55.97%	1164.51	0.08	0.00%	1164.51	0.08	0.00%
	8	1079.86	2.27	1704.62	0.001	57.86%	1079.85	0.08	0.00%	1079.85	0.09	0.00%
	9	1407.40	1.98	1766.55	0.001	25.52%	1407.40	0.08	0.00%	1407.40	0.08	0.00%
	10	1158.54	4.51	1853.87	0.001	60.02%	1158.53	0.07	0.00%	1158.53	0.07	0.00%
	Avg.	1126.45	2.14	1546.73	0.001	36.59%	1126.44	0.08	0.00%	1126.44	0.09	0.00%

15	1	1264.30	3600	1882.74	0.07	48.92%	1311.34	1.72	3.72%	1305.57	1.83	3.26%
	2	-	3600	1865.97	0.07	-	1608.60	0.62	-	1545.44	0.56	-
	3	1301.06	3600	2116.07	0.05	62.64%	1376.01	0.99	5.76%	1358.58	2.07	4.42%
	4	-	3600	3780.98	0.05	-	1682.68	1.09	-	1481.34	0.60	-
	5	-	3600	1888.71	0.56	-	1615.05	0.56	-	1615.05	0.44	-
	6	1306.63	3600	1772.24	0.03	35.63%	1348.41	0.63	3.20%	1340.48	0.66	2.59%
	7	1249.49	3600	2139.26	0.04	71.21%	1293.10	0.59	3.49%	1172.50	0.47	-6.16%
	8	1355.19	3600	3481.71	0.07	156.92%	1535.65	0.61	13.32%	1441.81	0.60	6.39%
	9	-	3600	2771.09	0.04	-	1714.82	0.60	-	1667.42	0.87	-
	10	-	3600	2489.43	0.06	-	1778.31	1.78	-	1694.46	1.32	-
	Avg.	1295.33	3600	2418.82	0.10	75.06%	1526.40	0.92	5.90%	1462.27	0.94	2.10%

1072

1073

Table 8 and Table 9 present the computation results on medium- and large-scale real-world instances,

1074

respectively. Similar to random cases, for these instances, Gurobi cannot obtain solutions within 3600

1075

seconds. Besides, C-ALNS continues to outperform greedy solutions by a large amount (-23.85% for

1076

medium-scale instances and -29.65% for large-scale instances). Moreover, the proposed RL-C-ALNS

1077

approach continues to deliver better results than C-ALNS with only 3 out of 40 instances that have no

1078

improvements, suggesting the effectiveness of the proposed RL component as well as the RL-C-ALNS

1079

approach.

1080

Table 8 Results on medium-scale real-world instances.

Instances		Gurobi	Greedy			C-ALNS			RL-C-ALNS			
		Value	CPU (s)	Value	CPU (s)	GAP ₁ (%)	Value	CPU (s)	GAP ₂ (%)	Value	CPU (s)	GAP ₃ (%)
25	1	-	3600	2764.93	0.16	-	2240.25	2.61	-18.98%	1909.50	2.88	-14.76%
	2	-	3600	4601.58	0.14	-	4064.02	2.11	-11.68%	1354.79	5.21	-66.66%
	3	-	3600	2913.91	0.11	-	2191.71	4.84	-24.78%	1822.34	2.41	-16.85%
	4	-	3600	2994.84	0.14	-	1837.33	3.21	-38.65%	1682.47	4.03	-8.43%
	5	-	3600	2535.57	0.14	-	2415.17	1.26	-4.75%	2415.17	1.74	0.00%
	6	-	3600	4562.32	0.13	-	2498.48	2.70	-45.24%	2266.23	2.79	-9.30%
	7	-	3600	3174.99	0.16	-	2476.55	1.70	-22.00%	2169.42	1.83	-12.40%
	8	-	3600	2970.37	0.14	-	2595.91	2.86	-12.61%	1811.55	4.48	-30.22%
	9	-	3600	2555.01	0.13	-	2474.71	1.43	-3.14%	2474.71	1.69	0.00%
	10	-	3600	2616.29	0.13	-	2083.98	3.23	-20.35%	2063.82	2.45	-0.97%
	Avg.	-	3600	3168.98	0.14	-	2487.81	2.60	-20.22%	1997.00	2.95	-15.96%
50	1	-	3600	7045.44	0.91	-	3535.20	37.83	-49.82%	3441.03	29.75	-2.66%
	2	-	3600	3726.10	1.09	-	2935.17	26.07	-21.23%	2325.27	31.50	-20.78%
	3	-	3600	4645.92	0.93	-	2123.96	46.55	-54.28%	2123.96	26.65	0.00%
	4	-	3600	4470.27	1.21	-	3488.22	35.19	-21.97%	2568.14	36.11	-26.38%
	5	-	3600	4214.49	0.89	-	3056.27	39.43	-27.48%	2632.92	25.11	-13.85%
	6	-	3600	3709.10	0.95	-	3175.40	23.09	-14.39%	2951.73	17.06	-7.04%
	7	-	3600	4504.53	0.90	-	3182.34	44.25	-29.35%	3172.04	28.68	-0.32%
	8	-	3600	4114.24	1.12	-	3652.80	30.37	-11.22%	2446.71	27.47	-33.02%
	9	-	3600	5718.86	0.99	-	3737.85	23.45	-34.64%	3525.09	24.85	-5.69%
	10	-	3600	4004.15	0.98	-	3587.56	16.92	-10.40%	2938.66	24.07	-18.09%
	Avg.	-	3600	4615.31	1.00	-	3247.48	32.32	-27.48%	2812.56	27.13	-12.78%

1081

Table 9 Results on large-scale real-world instances.

Instances	Gurobi	Greedy	C-ALNS	RL-C-ALNS
-----------	--------	--------	--------	-----------

		Value	CPU (s)	Value	CPU (s)	GAP ₁ (%)	Value	CPU (s)	GAP ₂ (%)	Value	CPU (s)	GAP ₃ (%)
75	1	-	3600	6787.85	3.45	-	3757.83	161.97	-44.64%	3757.83	171.38	0.00%
	2	-	3600	7598.27	3.42	-	4383.24	143.69	-42.31%	4063.16	107.72	-7.30%
	3	-	3600	7355.67	3.19	-	4181.23	102.73	-43.16%	2797.58	173.24	-33.09%
	4	-	3600	6343.21	4.47	-	3605.35	226.86	-43.16%	3607.49	159.72	0.06%
	5	-	3600	5564.26	4.11	-	4044.67	206.50	-27.31%	3971.60	127.51	-1.81%
	6	-	3600	6773.58	3.31	-	3660.56	179.40	-45.96%	3443.27	142.94	-5.94%
	7	-	3600	5547.15	4.63	-	4124.46	181.03	-25.65%	4009.46	194.41	-2.79%
	8	-	3600	6269.02	3.64	-	4190.75	144.60	-33.15%	4109.67	149.80	-1.93%
	9	-	3600	6933.23	3.60	-	5188.96	115.29	-25.16%	5135.15	146.27	-1.04%
	10	-	3600	8239.61	3.36	-	4940.26	152.04	-40.04%	4181.76	192.36	-15.35%
	Avg.	-	3600	6741.19	3.72	-	4207.73	161.41	-37.05%	3907.70	156.54	-6.92%
100	1	-	3600	12116.72	11.34	-	7823.40	482.71	-35.43%	7326.62	573.96	-6.35%
	2	-	3600	6752.46	9.50	-	6126.09	457.85	-9.28%	6030.51	479.66	-1.56%
	3	-	3600	9639.09	9.99	-	8236.79	677.44	-14.55%	7263.82	387.68	-11.81%
	4	-	3600	7188.93	9.71	-	6613.21	406.35	-8.01%	6691.01	359.42	1.18%
	5	-	3600	8184.75	11.33	-	7139.48	414.83	-12.77%	6967.27	392.68	-2.41%
	6	-	3600	10513.04	10.75	-	7346.39	469.00	-30.12%	7037.64	362.41	-4.20%
	7	-	3600	7746.63	9.41	-	6744.87	366.38	-12.93%	6651.75	405.87	-1.38%
	8	-	3600	7746.63	9.75	-	6888.14	518.92	-11.08%	6872.94	406.98	-0.22%
	9	-	3600	15495.63	9.37	-	8209.52	369.87	-47.02%	8092.15	386.21	-1.43%
	10	-	3600	13300.05	10.77	-	7799.06	432.65	-41.36%	7595.25	323.46	-2.61%
	Avg.	-	3600	9868.39	10.19	-	7292.69	459.60	-22.26%	7052.90	407.83	-3.08%

6.6. Sensitivity analysis

To balance between computational efficiency and instance scale while maintaining practicality, we select a real-world instance with 50 requests for sensitivity analysis. The four key parameters and their corresponding test ranges are summarized in Table 10. These parameters include the ratio of import requests among all three types of requests, the ratio of high-floor requests, the number of drones equipped by a van, and drone capacity, which jointly characterize demand structure and resource configuration in the TVDCTP-T. These parameters are selected as they correspond to key structural and operational factors of the TVDCTP-T. Specifically, the request mix ratio reflects demand composition, the high-floor ratio captures the level of mandatory drone usage, the number of drones represents fleet size decisions, and drone capacity characterizes resource capability. Together, these factors directly influence the interaction between truck-van-drone operations and thus are critical to evaluating the effectiveness of the proposed cascade-based solution approach.

Table 10 Parameters and test ranges for sensitivity analysis.

Parameter	Description	Range
Import ratio	demand composition	1:1:1 - 1:5:1
High-floor ratio	drone constraint level	10%-70%

Drone number	fleet size	1-5
Drone capacity	payload capacity	20-90 kg
Time-window tightness coefficient	temporal constraint level (controls time-window width via proportional scaling)	0.6-1.8

6.6.1. Sensitivity analysis results

Figure 5 presents how the objective value varies with four input factors. Overall, the objective exhibits non-linear responses to these parameters, indicating complex interactions among demand structure, resource allocation, and coordination decisions.

In Figure 5(a), changing the import-request ratio (Pick: Delivery: Both) yields a non-monotonic response: the curve rises from the first setting to 1:2:1, then drops to its lowest point around 1:3:1, increases slightly at 1:4:1, and finally climbs sharply at 1:5:1, which shows the largest objective value among the tested ratios.

Figure 5(b) shows a reversed U-shaped pattern with respect to the high-floor request ratio: the objective increases steadily from 0.1 to a peak around 0.4, and then decreases as the ratio continues to 0.7. A possible reason is that when the high-floor ratio is at a medium level, drones are already partly occupied by mandatory high-floor jobs, so drone capacity becomes tight but not to the point where the system can fully switch to a clear “drone-dominant” operating mode. At the same time, a large number of low-floor requests remain flexible and must be assigned between drones and vans. This creates a difficult trade-off: assigning more low-floor jobs to drones may reduce van detours, but it competes with the mandatory high-floor tasks; while delegating more low-floor jobs to vans avoids drone overload, but often increases van travel and reduces coordination efficiency. As a result, the algorithm needs frequent rebalancing between the two resources, which tends to produce more fragmented trips for both drones and vans, leading to the highest total cost. In

Figure 5(c), the objective generally increases with the number of drones, reaching a maximum around 4 drones, followed by a modest reduction at 5 drones. A possible reason is that adding drones introduces extra fixed dispatching costs, but brings low marginal savings once mandatory high-floor requests are already covered (leading to low utilization), and may fragment the assignment into many small drone trips, increasing coordination effort and reducing routing efficiency.

Figure 5(d) indicates a threshold-and-saturation effect for drone capacity: the objective decreases as

capacity grows from about 30 kg to roughly 60 kg (where the curve attains its minimum), and then slightly rebounds and levels off for larger capacities (around 70-90 kg), suggesting limited additional benefit.

Figure 5(e) shows a clear monotonic decreasing trend with respect to the time-window tightness coefficient. Here, the coefficient controls the width of each customer's time window by proportionally scaling it around its original midpoint, where a smaller value corresponds to tighter time windows and a larger value indicates more relaxed ones. As the coefficient increases, the objective value steadily decreases, indicating that relaxing time windows improves scheduling flexibility and facilitates better coordination among trucks, vans, and drones. However, the rate of improvement gradually diminishes at higher levels, suggesting that once temporal constraints are sufficiently relaxed, other factors such as vehicle capacity and system structure become the dominant limitations.

Overall, these results reveal three important characteristics of the system: (i) several parameters exhibit clear non-monotonic or threshold effects, while the time-window tightness coefficient shows a stable monotonic relationship, implying that inappropriate parameter settings may significantly deteriorate system performance whereas relaxing temporal constraints can consistently reduce cost; (ii) moderate configurations (e.g., balanced demand structures or medium resource levels) can sometimes lead to the worst performance due to increased coordination complexity; and (iii) beyond certain thresholds, increasing resources such as drone capacity or fleet size yields diminishing returns. These findings demonstrate that the proposed model is sensitive yet interpretable, and provide a solid basis for subsequent managerial insights.

6.6.2. Managerial implications

Based on Figure 5, the main managerial insights are that system cost is strongly affected by demand structure and fleet size, while supplementing more drones provides limited extra benefit.

First, Figure 5(a) shows that changing the mix ratio of Import / Export / Both requests does not lead to a monotonic up-or-down change in cost: a more balanced mix can be cheaper, but a highly unbalanced mix can push the cost much higher. This implies that managers should monitor the composition of requests in daily operations and, when possible, guide demand away from extreme mixes (e.g., through simple pricing controls, time-window guidance, or basic assignment rules).

Second, Figure 5(b) indicates that cost can be highest when the high-floor request ratio is in a middle range, so high-floor demand should be treated as a risk factor. When sensing the high-floor ratio is moving toward this costly range, managers may need stronger actions, such as tightening the service rules for drones,

or adjusting service promises to reduce late changes that create extra detours.

Third, Figure 5(c) suggests that deploying more drones does not reduce total cost: the curve rises as the number increases up to around four and only drops slightly at five. This means that additional drones may bring extra fixed costs and extra coordination effort, while the savings may not be enough to offset them. In practice, it can be better to keep a small base fleet and activate extra drones only when needed, rather than expanding the fleet permanently.

Fourth, Figure 5(d) shows a clear threshold for drone payload capacity: increasing capacity helps decrease cost up to about 60 kg, but the curve becomes almost flat beyond that. After reaching this threshold, further upgrades are less likely to cut costs, and resources may be better spent on improving operations, such as dispatch rules. Overall, Figure 5 suggests that managers can achieve larger cost reductions by focusing on Import/Export/Both demand structure and fleet-sizing policies, instead of relying mainly on hardware supplements.

Finally, Figure 5(e) shows that relaxing time-window constraints can consistently reduce system cost by improving scheduling flexibility and coordination efficiency. This suggests that overly strict service commitments may significantly increase operational cost, while allowing moderate flexibility in delivery time windows can lead to substantial improvements. However, excessively wide time windows bring only limited additional benefit, indicating that relaxing temporal constraints should be balanced with service quality considerations.

Overall, the findings suggest four key managerial principles: (i) demand structure management is crucial for avoiding inefficient operating regimes; (ii) resource sizing should prioritize flexibility over scale expansion; (iii) beyond certain thresholds, operational optimization is more valuable than additional hardware investment; and (iv) providing moderate flexibility in customer time windows can significantly reduce system cost, while overly strict service commitments should be avoided. These insights provide practical guidance for drone deployment, service mode design, and resource allocation in real-world applications.

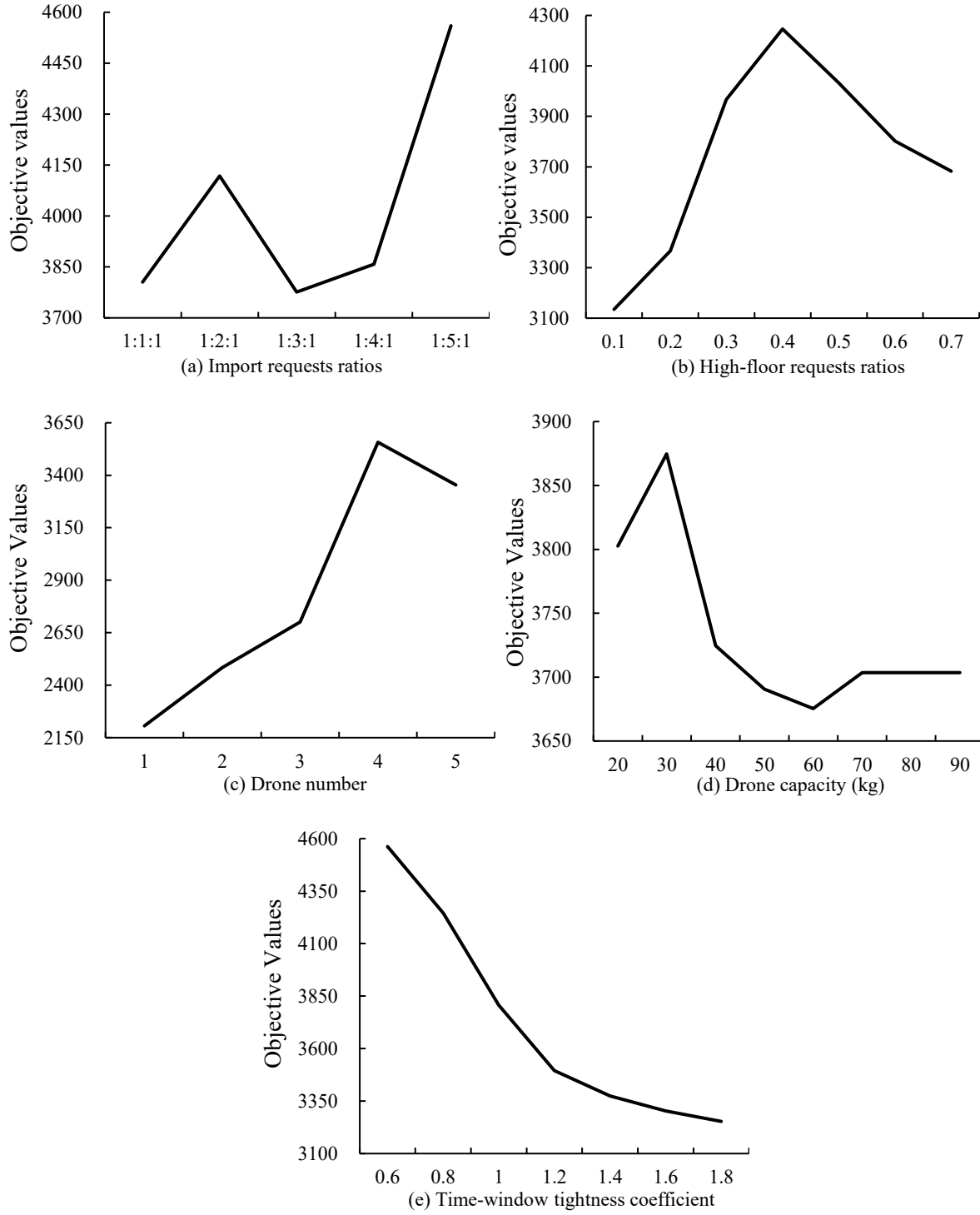


Figure 5 Impact of various factors on total cost: (a) import requests ratios, (b) high-floor requests ratios, (c) drone numbers, (d) drone capacity, (e) time-window tightness.

6.6.3. Stability and Scalability Analysis of RL-C-ALNS

To provide a more comprehensive evaluation of the proposed RL-C-ALNS algorithm, this section further analyzes its performance from two perspectives, namely stability and scalability, in addition to the solution quality and sensitivity analyses presented earlier. The objective is to examine the algorithm's

behavior under stochastic effects and increasing problem scales, thereby validating its robustness and practical applicability.

From the perspective of stability, we evaluate the performance of RL-C-ALNS under stochastic conditions by conducting multiple runs with different random seeds. The descriptive statistics of the results are summarized in Table 11 including the mean objective value, standard deviation, best and worst solutions, and the coefficient of variation (CV). The results indicate that the algorithm consistently exhibits low variability across all instance scales. Specifically, the CV values remain below 3% in all cases, suggesting that the solution quality is highly concentrated around the mean, with limited dispersion.

For small-scale instances, identical solutions are obtained across multiple runs, indicating that the search process converges to a stable pattern due to the relatively limited solution space. For larger instances, although minor variations are observed, the fluctuations remain well controlled, and no abnormal deviations occur. This demonstrates that, despite the stochastic nature of RL-C-ALNS, the final solutions do not rely on favorable random trajectories, but instead reflect a stable and robust search behavior.

In addition, it can be observed that the CV values in the Shenzhen instances are generally higher than those in the random instances, with a more pronounced increasing trend as the instance size grows. This can be attributed to the higher complexity of real-world road networks, where heterogeneous spatial distributions and more constrained routing choices amplify the variability introduced by stochastic search.

Building upon the above analysis, we further investigate the computational limits and scalability of RL-C-ALNS. As shown in Table 10 and Table 11, the computational time increases significantly with the problem size, especially when the number of requests exceeds 75. For random instances, the computation time is approximately 6-10 minutes for 75 requests and increases to about 10-13 minutes for 100 requests. For the Shenzhen instances, the computation time is approximately 2-4 minutes for 75 requests and rises to about 5-7 minutes for 100 requests. Furthermore, we conduct additional experiments on larger instances (e.g., 125 and 150 requests), where the computation time rapidly increases to 20 minutes and even up to one hour.

Overall, under the current experimental setting, RL-C-ALNS is capable of reliably solving medium-scale instances with up to 100 requests within a reasonable computational time. However, for larger-scale problems, the solution difficulty increases substantially due to the rapid expansion of the solution space and the structural complexity introduced by cascade operations, resulting in a sharp increase in computation time. Therefore, the proposed method is more suitable for offline optimization in medium-scale scenarios, such as

periodic logistics planning and scheduling, while further enhancements are required to improve its efficiency for large-scale or real-time applications.

Table 11 Descriptive statistics of RL-C-ALNS performance under different random seeds.

Instance		Mean objective	Std. dev.	MAX	MIN	Avg. CPU time(s)	CV (%)
Random	5	378.05	0.00	378.05	378.05	0.06	0.00%
	15	592.67	5.27	605.82	587.73	1.41	0.89%
	25	1023.01	15.76	1057.97	1006.95	4.78	1.54%
	50	2400.21	41.38	2422.10	2283.23	27.41	1.72%
	75	3542.13	52.56	3656.96	3476.30	339.68	1.48%
	100	5157.74	113.95	5386.06	4966.67	1111.29	2.21%
Shenzhen	5	1003.91	0.00	1003.91	1003.91	0.09	0.00%
	15	1402.18	20.57	1451.24	1371.53	1.96	1.47%
	25	2134.01	40.15	2180.22	2070.88	2.94	1.88%
	50	3266.53	76.06	3472.21	3204.10	35.95	2.33%
	75	4017.74	86.40	4131.23	3910.32	172.30	2.15%
	100	7432.73	141.74	7588.65	7203.71	376.88	1.91%

7. Conclusion

This study addresses a practical urban terminal transportation problem where commodities move from ports to warehouses and then to customers using a truck-van-drone system with warehouse transshipment. The model captures key operational rules and limits, including time windows and capacity constraints, building-level service requirements (high-floor requests served only by drones, low-floor requests served by vans or drones), and flexible docking between vans and drones. These features make the problem closer to real terminal delivery practice, but they also enlarge the search space and increase the need for scalable optimization.

To solve large instances, we propose RL-C-ALNS, which combines a cascade-based ALNS framework with an RL (PPO) module for operator selection. The cascade design links related route components so that changes in customer assignment trigger consistent updates of van routes and drone sub-routes, while the RL module learns to pick more useful destroy-repair pairs during different search stages. Importantly, RL is used only to guide operator choice; it does not change the neighborhood design, objective function, or simulated annealing acceptance rule. Computational results show three main points. First, on small instances the RL component may not take effect because the baseline search already reaches very satisfying solutions quickly. Second, on medium and large instances, RL-C-ALNS provides clear benefits in solution quality and often reduces runtime, especially when the problem is too large for exact solvers within a fixed time limit. Third,

the sensitivity plots provide practical guidance: cost can peak when the high-floor ratio is in a middle range because drones are partly locked by mandatory high-floor tasks while low-floor tasks still require careful split decisions between vans and drones; adding more drones can increase cost when extra fixed/coordination effort is larger than the saved travel; and increasing payload helps mainly up to a threshold, after which other limits dominate and further gains become small.

Future work can extend this setting by adding uncertain demand, weather-related drone limits, charging or battery swap decisions, and real-time re-optimization, and by testing more terminal networks and cost settings to further validate the method in practice. Furthermore, as more authoritative data sources become available, the formulation of drone-related fixed costs can be refined to explicitly incorporate practical components, such as maintenance, licensing, and operational overhead.

Appendix A. Formulation for the second stage

The second stage is triggered by the decisions in the first stage, and its formulation is presented in Eq. (110) - (135). After the van-drone fleet completes the second phase, the collected commodities are loaded into containers at warehouses waiting for container drayage, which is the second state. The second-stage model is the same as the container drayage part of the first stage, in which a tractor hauls a trailer to transport a container from transshipment warehouses to the port. Unlike the first stage, the only difference of the second stage compared to the container drayage stage is that the destinations of all containers in this stage $\tilde{d}_c$ are the port.

The second stage is formulated as a mixed-integer linear programming (MILP) model. The objective is to minimize the total cost incurred in this stage, including transportation costs, fixed costs, and time-related penalty costs. Please note that we omit detailed explanations of this formulation because it is the same as the container drayage phase in the first stage (see Section 4).

$$\min F = c_k \sum_{k \in K} \sum_{i \in N_0} \sum_{j \in N_0} \tilde{d}_{ij} * y_{ij}^k + \sum_{i \in K} \beta_i * \tilde{u}_i + \alpha \sum_{i \in N_s} \sum_{k \in K} (d_i^k - a_i^k) \quad (110)$$

$$\sum_{j \in N_0} y_{i,j}^k = \tilde{u}_k, \forall k \in K, i \in N_m \quad (111)$$

$$\sum_{j \in N_0} y_{j,i}^k = \tilde{u}_k, \forall k \in K, i \in N_m \quad (112)$$

$$\sum_{j \in N_0} y_{i,j}^k = \sum_{j \in N_0} y_{j,i}^k, \forall k \in K, i \in N_0 \quad (113)$$

$$\sum_{j \in N_0} \sum_{k \in K} u_{i,j}^{k,p} = \tilde{u}_p, \forall p \in P, i \in N_n \quad (114)$$

$$\sum_{j \in N_0} \sum_{k \in K} u_{j,i}^{k,p} = \tilde{u}_p, \forall p \in P, i \in N_n \quad (115)$$

$$\sum_{k \in K} \sum_{j \in N_0} u_{i,j}^{k,p} = \sum_{k \in K} \sum_{j \in N_0} u_{j,i}^{k,p}, \forall p \in P, i \in N_0 \quad (116)$$

$$u_{i,j}^{k,p} \leq y_{i,j}^k, \forall k \in K, p \in P, i, j \in N_0 \quad (117)$$

$$\sum_{j \in N_0} \sum_{i \in N_0} y_{i,j}^k * \tilde{d}_{ij} \leq \tilde{k}_k * \tilde{u}_k, \forall k \in K \quad (118)$$

$$\sum_{k \in K} \sum_{j \in N_0} \sum_{i \in N_0} u_{i,j}^{k,p} * \tilde{d}_{ij} \leq \tilde{k}_p * \tilde{u}_p, \forall p \in P \quad (119)$$

$$\sum_{p \in P} z_{cp} = 1, \forall c \in C \quad (120)$$

$$\sum_{i \in N_0} \sum_{k \in K} u_{i,o_c}^{k,p} = z_{cp}, \forall c \in C, p \in P \quad (121)$$

$$\sum_{k \in K} u_{o_c, \tilde{d}_c}^{k,p} = z_{cp}, \forall c \in C, p \in P \quad (122)$$

$$\sum_{j \in N_0} \sum_{k \in K} u_{\tilde{d}_c, j}^{k,p} \geq z_{cp}, \forall c \in C, k \in K, p \in P \quad (123)$$

$$a_j^k \geq d_i^k + \tau_{i,j}^k - M(1 - y_{i,j}^k), \forall k \in K, (i, j) \in A_1 \quad (124)$$

$$a_j^k \leq d_i^k + \tau_{i,j}^k + M(1 - y_{i,j}^k), \forall k \in K, (i, j) \in A_1 \quad (125)$$

$$d_j^k \geq a_j^k, \forall k \in K, j \in N_0 \quad (126)$$

$$a_{o_c}^k \geq \dot{e}_c - M(1 - \sum_{i \in N_0} \sum_{p \in P} u_{i,o_c}^{k,p}), \forall k \in K, c \in C \quad (127)$$

$$H_c \geq a_j^k + s_c - M(1 - u_{o_c, \tilde{d}_c}^{k,p}) \forall k \in K, p \in P, c \in C \quad (128)$$

$$d_i^k \geq H_c - M(1 - \sum_{j \in N_0} u_{\tilde{d}_c, j}^{k,p}) - M(1 - z_{cp}), \forall k \in K, p \in P, c \in C \quad (129)$$

$$y_{i,j}^k \in \{0,1\}, i, j \in N_0, k \in K \quad (130)$$

$$u_{i,j}^{k,p} \in \{0,1\}, i, j \in N_0, k \in K, p \in P \quad (131)$$

$$\tilde{u}_i \in \{0,1\}, i \in K \quad (132)$$

$$z_{cp} \in \{0,1\}, c \in C, p \in P \quad (133)$$

$$H_c \geq 0, c \in C \quad (134)$$

$$a_i^k, d_i^k \geq 0, i \in N_0, k \in K \quad (135)$$

Appendix B. Hyperparameters for RL-C-ALNS

Table 12 Hyperparameters for RL-C-ALNS.

Parameter and description	Value
Maximum number of iterations	1000
Customer removal ratio	0.2

Initial temperature in simulated annealing	1000
Cooling rate	0.9995
Length of the interval for updating operator weights	50
Weight reaction coefficient	0.2
Maximum number of consecutive non-improving iterations	100
Discount factor	0.95
PPO clipping threshold	0.2
Learning rate of the policy network	1e-3
Learning rate of the value network	1e-3
Batch size per update	32
Number of epochs per policy update	10

References

- Achamrah, F. E., Riane, F., & Limbourg, S. (2022). Solving inventory routing with transshipment and substitution under dynamic and stochastic demands using genetic algorithm and deep reinforcement learning. *International Journal of Production Research*, 60(20), 6187-6204. <https://doi.org/10.1080/00207543.2021.1987549>
- Agatz, N., Bouman, P., & Schmidt, M. (2018). Optimization Approaches for the Traveling Salesman Problem with Drone. *Transportation Science*, 52(4), 965-981. <https://doi.org/10.1287/trsc.2017.0791>
- Alfandari, L., Ljubic, I., & da Silva, M. D. (2022). A tailored Benders decomposition approach for last-mile delivery with autonomous robots. *European Journal of Operational Research*, 299(2), 510-525. <https://doi.org/10.1016/j.ejor.2021.06.048>
- Avci, M., & Yildiz, S. T. (2020). A mathematical programming-based heuristic for the production routing problem with transshipments. *Computers & Operations Research*, 123, Article 105042. <https://doi.org/10.1016/j.cor.2020.105042>
- CAAM. (2024). Annual Report on China Commercial Vehicle Market. <http://cv.ce.cn/zt/2025/2024pd/index.shtml>
- Carlsson, J. G., & Song, S. Y. (2018). Coordinated Logistics with a Truck and a Drone. *Management Science*, 64(9), 4052-4069. <https://doi.org/10.1287/mnsc.2017.2824>
- Chen, C., Demir, E., Huang, Y., & Qiu, R. Z. (2021). The adoption of self-driving delivery robots in last mile logistics. *Transportation Research Part E-Logistics and Transportation Review*, 146, Article 102214. <https://doi.org/10.1016/j.tre.2020.102214>
- Chen, S. K., Wang, H., & Meng, Q. (2021). Autonomous truck scheduling for container transshipment between two seaport terminals considering platooning and speed optimization. *Transportation Research Part B-Methodological*, 154, 289-315. <https://doi.org/10.1016/j.trb.2021.10.014>
- Chen, X. C., He, S. W., Zhang, Y. X., Tong, L., Shang, P., & Zhou, X. S. (2020). Yard crane and AGV scheduling in automated container terminal: A multi-robot task allocation framework. *Transportation Research Part C-Emerging Technologies*, 114, 241-271. <https://doi.org/10.1016/j.trc.2020.02.012>
- Cheng, S., Liu, Q. Y., Jin, H., Zhang, R., Ma, L. H., & Kwong, C. F. (2025). Collaborative optimization of truck scheduling in container terminals using graph theory and DDQN. *Scientific Reports*, 15(1), Article 6950. <https://doi.org/10.1038/s41598->

- 025-91140-7
- Dayarian, I., Savelsbergh, M., & Clarke, J. P. (2020). Same-Day Delivery with Drone Resupply. *Transportation Science*, 54(1), 229-249. <https://doi.org/10.1287/trsc.2019.0944>
- De Rosa, B., Improta, G., Ghiani, G., & Musmanno, R. (2002). The arc routing and scheduling problem with transshipment. *Transportation Science*, 36(3), 301-313. <https://doi.org/10.1287/trsc.36.3.301.7832>
- Dellaert, N., Dashty Saridarq, F., Van Woensel, T., & Crainic, T. G. (2019). Branch-and-Price-Based Algorithms for the Two-Echelon Vehicle Routing Problem with Time Windows. *Transportation Science*, 53(2), 463-479. <https://doi.org/10.1287/trsc.2018.0844>
- Dorling, K., Heinrichs, J., Messier, G. G., & Magierowski, S. (2017). Vehicle Routing Problems for Drone Delivery. *Ieee Transactions on Systems Man Cybernetics-Systems*, 47(1), 70-85. <https://doi.org/10.1109/tsmc.2016.2582745>
- Duan, H. H., Li, X. L., Feng, Y. X., Zhang, G. H., Lu, Q. C., & Meng, X. (2026). Multi-objective hyper heuristic memetic algorithm for time-dependent vehicle routing problem with drone considering simultaneous pickup and delivery. *Expert Systems with Applications*, 297, Article 129196. <https://doi.org/10.1016/j.eswa.2025.129196>
- Ensafian, H., Andaryan, A. Z., Bell, M. G. H., Geers, D. G., Kilby, P., & Li, J. (2023). Cost-optimal deployment of autonomous mobile lockers co-operating with couriers for simultaneous pickup and delivery operations. *Transportation Research Part C-Emerging Technologies*, 146, Article 103958. <https://doi.org/10.1016/j.trc.2022.103958>
- Fan, H., Liu, K., Wang, J. B., & Zhao, J. (2026). Multi-depot electric vehicle routing problem with drones considering time-dependent traffic. *Advanced Engineering Informatics*, 69, Article 104008. <https://doi.org/10.1016/j.aei.2025.104008>
- Friedrich, C., & Elbert, R. (2022). Adaptive large neighborhood search for vehicle routing problems with transshipment facilities arising in city logistics. *Computers & Operations Research*, 137, Article 105491. <https://doi.org/10.1016/j.cor.2021.105491>
- Ghiasvand, M. R., Rahmani, D., & Moshref-Javadi, M. (2024). Data-driven robust optimization for a multi-trip truck-drone routing problem. *Expert Systems with Applications*, 241, Article 122485. <https://doi.org/10.1016/j.eswa.2023.122485>
- Gonzalez-R, P. L., Canca, D., Andrade-Pineda, J. L., Calle, M., & Leon-Blanco, J. M. (2020). Truck-drone team logistics: A heuristic approach to multi-drop route planning. *Transportation Research Part C-Emerging Technologies*, 114, 657-680. <https://doi.org/10.1016/j.trc.2020.02.030>
- Hou, Y., Shen, Y. J., Han, H. G., & Wang, J. J. (2025). Multi-task differential evolution algorithm with dynamic resource allocation: A study on e-waste recycling vehicle routing problem ☆. *Swarm and Evolutionary Computation*, 92, Article 101806. <https://doi.org/10.1016/j.swevo.2024.101806>
- Institute for Employment and Livelihood Studies, Renmin University of China. (2025). *Survey Report on Employment and Living Conditions of Truck Drivers in China*. <https://www.workercn.cn/c/2025-08-21/8584059.shtml>
- Jia, S., Cui, H. P., Chen, R., & Meng, Q. (2022). Dynamic container drayage with uncertain request arrival times and service time windows. *Transportation Research Part B: Methodological*, 166, 237-258. <https://doi.org/10.1016/j.trb.2022.10.010>
- Jiang, J., Dai, Y., Yang, F., & Ma, Z. J. (2024). A multi-visit flexible-docking vehicle routing problem with drones for simultaneous pickup and delivery services. *European Journal of Operational Research*, 312(1), 125-137. <https://doi.org/10.1016/j.ejor.2023.06.021>
- Jiang, Y., Liu, M. M., Jia, X. B., & Xue, Q. W. (2025). The multi-visit vehicle routing problem with multiple heterogeneous drones. *Transportation Research Part C-Emerging Technologies*, 172, Article 105026. <https://doi.org/10.1016/j.trc.2025.105026>
- Kang, M., & Lee, C. (2021). An Exact Algorithm for Heterogeneous Drone-Truck Routing Problem. *Transportation Science*, 55(5), 1088-1112. <https://doi.org/10.1287/trsc.2021.1055>
- Karak, A., & Abdelghany, K. (2019). The hybrid vehicle-drone routing problem for pick-up and delivery services. *Transportation Research Part C-Emerging Technologies*, 102, 427-449. <https://doi.org/10.1016/j.trc.2019.03.021>
- Kong, J. L., Xie, M. H., & Wang, H. (2025). Integrating Autonomous Vehicles and Drones for Last-Mile Delivery: A Routing

- Problem with Two Types of Drones and Multiple Visits. *Drones*, 9(4), Article 280. <https://doi.org/10.3390/drones9040280>
- Kuo, R., Edbert, E., Zulvia, F. E., & Lu, S. H. (2023). Applying NSGA-II to vehicle routing problem with drones considering makespan and carbon emission. *Expert Systems with Applications*, 221, Article 119777. <https://doi.org/10.1016/j.eswa.2023.119777>
- Lehmann, J., & Winkenbach, M. (2024). A matheuristic for the Two-Echelon Multi-Trip Vehicle Routing Problem with mixed pickup and delivery demand and time windows. *Transportation Research Part C: Emerging Technologies*, 160. <https://doi.org/10.1016/j.trc.2024.104522>
- Li, J., Cang, L., Wu, Y. S., & Zhang, Z. T. (2025). Two-echelon collaborative many-to-many pickup and delivery problem for agricultural wholesale markets with workload balance. *Omega-International Journal of Management Science*, 130, Article 103164. <https://doi.org/10.1016/j.omega.2024.103164>
- Li, J., Ensafian, H., Bell, M. G. H., & Geers, D. G. (2021). Deploying autonomous mobile lockers in a two-echelon parcel operation. *Transportation Research Part C-Emerging Technologies*, 128, Article 103155. <https://doi.org/10.1016/j.trc.2021.103155>
- Li, J. H., & Wang, M. (2025). A New Vehicle-Multi-Drone Collaborative Delivery Path Optimization Approach. *Symmetry-Basel*, 17(9), Article 1382. <https://doi.org/10.3390/sym17091382>
- Li, K., Cui, H., Meng, Q., & Zhang, X. (2025). Dynamic truck-drone collaborative transportation during hurricanes. *Transportation Research Part C: Emerging Technologies*, 180. <https://doi.org/10.1016/j.trc.2025.105322>
- Liang, Y. J., & Luo, Z. X. (2022). A Survey of Truck-Drone Routing Problem: Literature Review and Research Prospects. *Journal of the Operations Research Society of China*, 10(2), 343-377. <https://doi.org/10.1007/s40305-021-00383-4>
- Liu, X., Chung, S. H., & Kwon, C. (2025). An adaptive large neighborhood search method for the drone-truck arc routing problem. *Computers & Operations Research*, 176. <https://doi.org/10.1016/j.cor.2024.106959>
- Liu, Y., Liu, Z., Shi, J., Wu, G., & Pedrycz, W. (2021). Two-Echelon Routing Problem for Parcel Delivery by Cooperated Truck and Drone. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 51(12), 7450-7465. <https://doi.org/10.1109/tsmc.2020.2968839>
- Luo, Z. H., Poon, M., Zhang, Z. Z., Liu, Z., & Lim, A. (2021). The Multi-visit Traveling Salesman Problem with Multi-Drones. *Transportation Research Part C-Emerging Technologies*, 128, Article 103172. <https://doi.org/10.1016/j.trc.2021.103172>
- Morim, A., Campuzano, G., Amorim, P., Mes, M., & Lalla-Ruiz, E. (2024). The drone-assisted vehicle routing problem with robot stations. *Expert Systems with Applications*, 238, Article 121741. <https://doi.org/10.1016/j.eswa.2023.121741>
- Murray, C. C., & Chu, A. G. (2015). The flying sidekick traveling salesman problem: Optimization of drone-assisted parcel delivery. *Transportation Research Part C-Emerging Technologies*, 54, 86-109. <https://doi.org/10.1016/j.trc.2015.03.005>
- Nguyen, N. N. T., & Hop, N. V. (2025). Hierarchical allocation-routing heuristic algorithm for crowd-shipping problem with time windows, transshipment nodes, and delivery options. *Expert Systems with Applications*, 268, Article 126325. <https://doi.org/10.1016/j.eswa.2024.126325>
- Ostermeier, M., Heimfarth, A., & Hübner, A. (2022). Cost-optimal truck-and-robot routing for last-mile delivery. *Networks*, 79(3), 364-389. <https://doi.org/10.1002/net.22030>
- Peng, W. H., Wang, D. J., Yin, Y. Q., & Cheng, T. C. E. (2025). Multi-agent deep reinforcement learning-based truck-drone collaborative routing with dynamic emergency response. *Transportation Research Part E-Logistics and Transportation Review*, 195, Article 103974. <https://doi.org/10.1016/j.tre.2025.103974>
- Peng, Y., Huang, J., Yu, D. Z., Tanksale, A., & Gu, P. F. (2026). Deep reinforcement learning for drone-assisted truck delivery route optimization in a time-dependent network with time windows. *Computers & Industrial Engineering*, 211, Article 111624. <https://doi.org/10.1016/j.cie.2025.111624>
- Perboli, G., Tadei, R., & Vigo, D. (2011). The Two-Echelon Capacitated Vehicle Routing Problem: Models and Math-Based Heuristics. *Transportation Science*, 45(3), 364-380. <https://doi.org/10.1287/trsc.1110.0368>
- Poikonen, S., & Golden, B. (2020). Multi-visit drone routing problem. *Computers & Operations Research*, 113, Article 104802.

- <https://doi.org/10.1016/j.cor.2019.104802>
- Ren, X. X., Fan, H. M., Fan, H., & Ma, M. Z. (2025). Time-dependent electric vehicle-drone routing problem for scheduled deliveries and on-demand pickups considering variable drone speeds and no-fly zones. *Computers & Operations Research*, 182, Article 107141. <https://doi.org/10.1016/j.cor.2025.107141>
- Ropke, S., & Pisinger, D. (2006). An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows. *Transportation Science*, 40(4), 455-472. <https://doi.org/10.1287/trsc.1050.0135>
- Salama, M. R., & Srinivas, S. (2022). Collaborative truck multi-drone routing and scheduling problem: Package delivery with flexible launch and recovery sites. *Transportation Research Part E-Logistics and Transportation Review*, 164, Article 102788. <https://doi.org/10.1016/j.tre.2022.102788>
- Saleu, R. G. M., Deroussi, L., Feillet, D., Grangeon, N., & Quilliot, A. (2022). The parallel drone scheduling problem with multiple drones and vehicles. *European Journal of Operational Research*, 300(2), 571-589. <https://doi.org/10.1016/j.ejor.2021.08.014>
- Shen, S. Y., & Honda, M. (2009). Incorporating lateral transfers of vehicles and inventory into an integrated replenishment and routing plan for a three-echelon supply chain. *Computers & Industrial Engineering*, 56(2), 754-775. <https://doi.org/10.1016/j.cie.2007.09.004>
- Triantali, D. G., Parsopoulos, K. E., & Skouri, K. (2025). Data-driven approach for cost-optimized last-mile delivery with transshipment nodes and occasional drivers. *Journal of Global Optimization*. <https://doi.org/10.1007/s10898-025-01553-x>
- Wang, X. Y., Poikonen, S., & Golden, B. (2017). The vehicle routing problem with drones: several worst-case results. *Optimization Letters*, 11(4), 679-697. <https://doi.org/10.1007/s11590-016-1035-3>
- Xiao, Q. X., & Gao, J. J. (2024). The Multi-Visit Vehicle Routing Problem with Drones under Carbon Trading Mechanism. *Sustainability*, 16(14), Article 6145. <https://doi.org/10.3390/su16146145>
- Xing, J. H., Guo, T., & Tong, L. (2024). Reliable truck-drone routing with dynamic synchronization: A high-dimensional network programming approach. *Transportation Research Part C-Emerging Technologies*, 165, Article 104698. <https://doi.org/10.1016/j.trc.2024.104698>
- Yu, S. H., Puchinger, J., & Sun, S. D. (2020). Two-echelon urban deliveries using autonomous vehicles. *Transportation Research Part E-Logistics and Transportation Review*, 141, Article 102018. <https://doi.org/10.1016/j.tre.2020.102018>
- Yu, V. F., Jodiawan, P., Schrottenboer, A. H., & Hou, M. L. (2023). The two-echelon vehicle routing problem with time windows, intermediate facilities, and occasional drivers. *Expert Systems with Applications*, 234, Article 120945. <https://doi.org/10.1016/j.eswa.2023.120945>
- Yu, V. F., Nguyen, M. P. K., Putra, K., Gunawan, A., & Dharma, I. (2022). The Two-Echelon Vehicle Routing Problem with Transshipment Nodes and Occasional Drivers: Formulation and Adaptive Large Neighborhood Search Heuristic. *Journal of Advanced Transportation*, 2022, Article 5603956. <https://doi.org/10.1155/2022/5603956>
- Zeng, J., Hu, Y., & Pei, M. (2026). Optimizing bidirectional delivery with multiple drones and trucks: a mixed-integer nonlinear model to addressing no-fly zone constraints. *Transportation Research Part E: Logistics and Transportation Review*, 205. <https://doi.org/10.1016/j.tre.2025.104511>
- Zhang, G. W., Zhu, N., Ma, S. F., & Xia, J. (2021). Humanitarian relief network assessment using collaborative truck-and-drone system. *Transportation Research Part E-Logistics and Transportation Review*, 152, Article 102417. <https://doi.org/10.1016/j.tre.2021.102417>
- Zhen, L., Gao, J., Wang, S., Laporte, G., & Yue, X. (2025). Optimizing an On-Demand Delivery Mode Based on Trucks and Drones. *Transportation Science*, 59(5), 1008-1031. <https://doi.org/10.1287/trsc.2024.0693>
- Zhou, J., Yi, J., Yang, Z. Y., Pu, H. Y., Li, X. Y., Luo, J., & Gao, L. (2025). A survey on vehicle-drone cooperative delivery operations optimization: Models, methods, and future research directions. *Swarm and Evolutionary Computation*, 92, Article 101780. <https://doi.org/10.1016/j.swevo.2024.101780>

